

# HÀNH NGHỀ VÀ LUYỆN TẬP

---



## TÓM TẮT

Phần E là khoảng cách giữa “tôi biết phải làm gì” và “chương trình chạy đúng, đúng hạn”. Chương 25 nói về cài đặt, gỡ lỗi và kiểm thử — trong đó stress test là kỹ thuật đáng giá nhất mà rất nhiều người học muộn. Chương 26 nói về thuật toán trong hệ thống thật, nơi hằng số, bộ nhớ và tính bảo trì thường thắng độ phức tạp tiệm cận. Chương 27 là chiến thuật phỏng vấn và thi đấu: quản lý thời gian, chọn bài, nói ra suy nghĩ. Chương 28 là lộ trình luyện tập và ngân hàng bài tập lớn để mang theo sau khi gấp sách. Nếu chỉ nhớ một điều: năng lực giải thuật tăng theo số bài bạn *giải xong sau khi đã bí*, không theo số bài bạn đọc lời giải.

Phần này tồn tại vì một quan sát khó chịu: rất nhiều người hiểu thuật toán tốt hơn hẳn kết quả họ đạt được. Nguyên nhân gần như luôn nằm ở bốn chỗ, và bốn chương của phần E ứng đúng với bốn chỗ đó. Thứ nhất, code có bug mà không có quy trình tìm bug — chương 25. Thứ hai, chọn sai thứ để tối ưu trong bối cảnh thật, ví dụ đổi cấu trúc dữ liệu để giảm độ phức tạp trong khi nút cổ chai nằm ở truy cập bộ nhớ — chương 26. Thứ ba, chiến thuật kém: dành 50 phút cho bài khó nhất rồi không kịp làm hai bài dễ — chương 27. Thứ tư, luyện tập không có cấu trúc: giải nhiều mà không tiến, vì luôn giải những bài vừa sức thay vì những bài hơi quá sức — chương 28.

Ba chương đầu ngắn và thực dụng; chương 28 dài vì nó chứa ngân hàng bài tập được phân loại theo chương, kèm gợi ý và lộ trình theo tuần. Đó là chương bạn sẽ mở lại nhiều lần nhất sau khi đã đọc xong cả quyển.

## Bản đồ phần E — Hành nghề và luyện tập

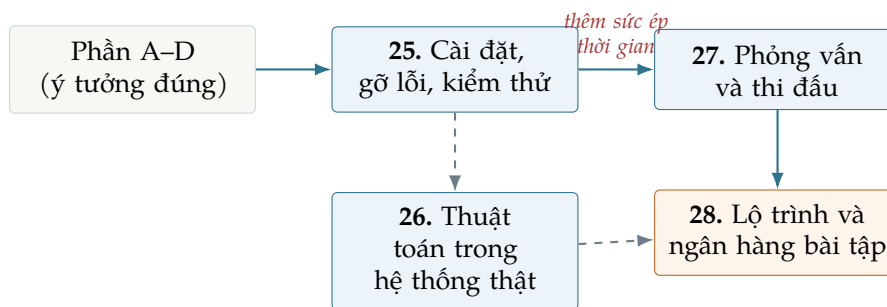
## TÓM TẮT

Bốn chương ứng với bốn chỗ hay hổng giữa “hiểu thuật toán” và “có kết quả”: bug không tìm được (25), tối ưu sai chỗ trong hệ thống thật (26), chiến thuật kém khi có giới hạn thời gian (27), và luyện tập không có cấu trúc (28). Không có phụ thuộc bắt buộc giữa bốn chương; đọc theo nhu cầu. Chương 28 là chương dùng lại nhiều nhất về lâu dài.

## A. Phần này có gì

- **25-cai-dat-go-loi-kiem-thu** — viết code đúng ngay từ đầu, danh mục bug kinh điển, khẳng định và bất biến trong code, stress test bằng sinh test ngẫu nhiên, thu nhỏ phân ví dụ, đo thời gian.
- **26-thuat-toan-trong-he-thong-that** — vì sao  $O(n \log n)$  chậm hơn  $O(n^2)$  ở  $n$  nhỏ, chi phí bộ nhớ và cache, đo trước khi tối ưu, thuật toán trong cơ sở dữ liệu, trình biên dịch, mạng, đồ hoạ và robot.
- **27-phong-van-va-thi-dau** — quy trình 45 phút của một buổi phỏng vấn, cách nói ra suy nghĩ, chọn bài và phân bổ thời gian trong contest, xử lý lúc bí, đọc bảng điểm.
- **28-lo-trinh-va-ngan-hang-bai-tap** — lộ trình 12 tuần và lộ trình 6 tháng, nguyên tắc up-solving, nhật ký lỗi, lập lại ngắt quãng, và ngân hàng bài tập phân loại theo 27 chương trước.

## B. Phụ thuộc và thứ tự đọc



**Hình 1.** Phần E nhận đầu vào từ toàn bộ phần A–D và chia làm hai nhánh: nhánh kỹ thuật (25–26) và nhánh chiến thuật cùng luyện tập (27–28). Chương 28, tô hồ phách, là nơi mọi thứ quay về thành lịch luyện cụ thể.

## C. Cốt lõi hay tham khảo

**Cốt lõi:** stress test và danh mục bug ở chương 25; nguyên tắc “đo trước khi tối ưu” ở chương 26; quy trình nói ra suy nghĩ ở chương 27; nguyên tắc upsolving và nhật ký lỗi ở chương 28. **Tham khảo:** các mục khảo sát thuật toán trong từng ngành công nghiệp ở chương 26 — đọc để biết bức tranh, không cần thuộc.

Phần thực hành bổ sung: [E/25](#) có bảng chọn lớp kiểm thử và phân biệt chia đôi với thu nhỏ phần ví dụ; [E/26](#) có phiếu benchmark và phép tính giới hạn tăng tốc toàn pipeline.

# Cài đặt, gỡ lỗi và kiểm thử (Implementation, Debugging, Testing)

**PHỤ THUỘC** A/04 cho bất biến — công cụ phòng lỗi mạnh nhất; C/13 cho lời giải vét cạn, thứ đóng vai trò trọng tài trong stress test.

## TÓM TẮT

Chương này nói về khoảng cách giữa “tôi biết phải làm gì” và “chương trình chạy đúng”. Ba phần: viết code sao cho ít lỗi ngay từ đầu — chủ yếu bằng cách viết bất biến trước khi viết vòng lặp; danh mục các lỗi kinh điển kèm cách phòng từng loại; và quan trọng nhất, một quy trình gỡ lỗi có hệ thống thay cho việc sửa mò. Trung tâm của quy trình đó là *stress test*: sinh dữ liệu nhỏ ngẫu nhiên, chạy song song lời giải nhanh và lời giải vét cạn, so kết quả, rồi thu nhỏ phản ví dụ. Nếu chỉ nhớ một điều: khi bạn không biết vì sao code sai, đừng đọc lại code — hãy để máy tìm cho bạn một phản ví dụ nhỏ, vì mười phút viết bộ sinh test gần như luôn rẻ hơn một giờ nhìn chằm chằm màn hình.

## KIẾN THỨC CẦN CÓ TRƯỚC

- **stress test** — chạy hai lời giải — một nhanh, một chậm nhưng chắc đúng — trên hàng nghìn dữ liệu nhỏ sinh ngẫu nhiên, và dừng lại ở lần đầu hai kết quả khác nhau.
- **thu nhỏ phản ví dụ** — từ một dữ liệu làm chương trình sai, tìm dữ liệu nhỏ hơn vẫn giữ được lỗi — vì lỗi trên dữ liệu ba phần tử dễ hiểu hơn nhiều so với lỗi trên ba nghìn phần tử.
- **khẳng định (assertion)** — câu lệnh kiểm tra một điều kiện phải đúng tại một điểm trong chương trình; nếu sai thì dừng ngay tại chỗ phát sinh thay vì để lỗi lan ra.
- **lỗi im lặng** — lỗi không làm chương trình sập mà chỉ làm kết quả sai — loại nguy hiểm nhất, vì nó chỉ lộ ra khi có người đối chiếu.
- **dữ liệu biên** — các trường hợp ở ranh giới: rỗng, một phần tử, mọi phần tử bằng nhau, giá trị lớn nhất, giá trị âm.
- **tái lập** — khả năng chạy lại và gặp đúng lỗi cũ; với chương trình có ngẫu nhiên, điều này đòi phải ghi lại hạt giống.

## CÂU HỎI MỞ ĐẦU

- Q1. Vì sao “đọc lại code” là chiến lược gỡ lỗi kém, dù đó là việc ai cũng làm đầu tiên?
- Q2. Stress test cần một lời giải vét cạn — nghe như làm gấp đôi công việc. Vì sao nó vẫn tiết kiệm thời gian?
- Q3. Vì sao lỗi trên dữ liệu nhỏ lại dễ sửa hơn lỗi trên dữ liệu lớn tới mức đáng bỏ công thu nhỏ phản ví dụ?
- Q4. Loại lỗi nào sống sót qua mọi bộ test ngẫu nhiên, và phải phòng bằng cách khác?
- Q5. Khi chương trình có yếu tố ngẫu nhiên, làm sao gỡ lỗi được?

Có một cảnh quen thuộc: bạn nộp bài, nhận về “sai ở test 12”, và bắt đầu đọc lại code. Đọc ba lần, mọi dòng đều hợp lý. Bạn đổi một dấu < thành <=, nộp lại, vẫn sai. Đổi lại như cũ, thêm một trường hợp riêng, vẫn sai. Nửa giờ trôi qua và bạn không biết mình đang tiến hay lùi.

Vấn đề của cảnh này không phải kỹ năng lập trình mà là *thiếu quy trình*. Đọc lại code là cách gỡ lỗi kém vì hai lý do: bạn đọc lại chính suy nghĩ của mình, nên bạn sẽ đọc thấy điều bạn *định* viết chứ không phải điều bạn *đã* viết; và bạn không có thông tin mới nào — đúng tình trạng “lặp lại một biểu diễn” mà chương A/01 đã mô tả.

Chương này thay cảnh đó bằng một quy trình. Quy trình ấy dựa trên một nguyên tắc duy nhất: *hãy để máy tìm cho bạn một ví dụ sai nhỏ, rồi mới dùng đầu.*

## 1.1 Phòng bệnh: viết code ít lỗi ngay từ đầu

---

Cách rẻ nhất để sửa lỗi là không tạo ra nó. Bốn thói quen có hiệu quả rõ rệt.

*Viết bất biến trước khi viết vòng lặp.* Đây là lời khuyên của chương A/04 và nó thuộc về chương này nhiều nhất. Một dòng chú thích — “sau vòng lặp thứ  $i$ ,  $best$  là đáp án của  $a[0..i]$ ” — biến mọi quyết định về chỉ số từ đoán mò thành suy diễn. Phần lớn lỗi lệch một đơn vị chết ngay tại đây.

*Chọn quy ước và giữ nhất quán.* Đánh chỉ số từ 0 hay từ 1, đoạn đóng  $[l, r]$  hay nửa mở  $[l, r)$  — chọn một và dùng cho toàn bài. Phần lớn lỗi biên đến từ việc trộn hai quy ước trong cùng một chương trình. Nửa mở là quy ước ít gây lỗi hơn cho các bài chia đoạn, vì độ dài bằng đúng  $r - l$  và phép chia đôi không có trường hợp riêng.

*Ưu tiên rõ ràng hơn ngắn.* Một dòng code khéo léo tiết kiệm ba dòng nhưng tốn hai mươi phút khi phải gỡ lỗi. Trong thi đấu điều này còn quan trọng hơn ngoài đời, vì bạn không có thời gian đọc lại thứ mình không hiểu ngay.

*Đặt khẳng định ở những chỗ bạn tin chắc.* Nghịch lý là ở chỗ: nơi đáng đặt khẳng định nhất chính là nơi bạn *chắc chắn* nó luôn đúng, vì đó là những giả định ngầm mà bạn sẽ không bao giờ nghĩ tới khi gỡ lỗi. Khẳng định làm lỗi dừng lại tại nơi phát sinh thay vì để nó lan ra và chỉ lộ ở kết quả cuối.

## 1.2 Danh mục lỗi kinh điển

---

Phần lớn lỗi trong lập trình thuật toán rơi vào một số ít loại. Thuộc danh mục này giúp bạn kiểm nhanh trước khi nộp, và cột thứ ba là cách phòng chứ không phải cách sửa.

| Loại lỗi                        | Dấu hiệu                                         | Cách phòng                                                                      |
|---------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------|
| Tràn số                         | Kết quả âm bất ngờ, hoặc sai chỉ với dữ liệu lớn | Ước lượng giá trị lớn nhất <i>trước</i> ; mặc định dùng 64 bit cho tổng và tích |
| Lệch một đơn vị                 | Sai đúng ở phần tử đầu hoặc cuối                 | Viết bất biến; chọn một quy ước đoạn và giữ nhất quán                           |
| Chưa khởi tạo                   | Kết quả đổi giữa các lần chạy                    | Khởi tạo tường minh; đặt khẳng định về miền giá trị hợp lệ                      |
| Hàm so sánh không nhất quán     | Sắp xếp cho kết quả lạ, đôi khi sập              | Kiểm quan hệ có bắc cầu; không dùng “gần bằng” làm khoá sắp xếp                 |
| Đệ quy quá sâu                  | Sập với dữ liệu lớn, chạy tốt với dữ liệu nhỏ    | Ước lượng độ sâu tối đa; chuyển sang bản lặp khi cần                            |
| Chia số nguyên                  | Sai với số âm hoặc mất phần lẻ                   | Nhân chéo thay vì chia; cẩn thận với hướng làm tròn của số âm                   |
| Phần dư âm                      | Kết quả âm sau phép trừ modulo                   | Cộng thêm $m$ rồi lấy dư lần nữa $(D/21)$                                       |
| Trường hợp $n = 0$ hoặc $n = 1$ | Sai đúng ở test đầu hoặc test biên               | Luôn thử tay hai ca này trước khi nộp                                           |
| Đọc dữ liệu chậm                | Quá thời gian dù thuật toán đúng bậc             | Dùng cách đọc nhanh khi $n$ tới hàng triệu                                      |

Có một lỗi thứ mười không nằm trong bảng vì nó thuộc loại khác: *hiểu sai đề*. Nó chiếm tỉ lệ cao hơn nhiều người thừa nhận, và cách phòng duy nhất là bước 1 của Pólya — phát biểu lại đề bằng lời của mình và tính tay ví dụ trong đề (A/01).

### 1.3 Stress test: kỹ thuật đánh giá nhất chương

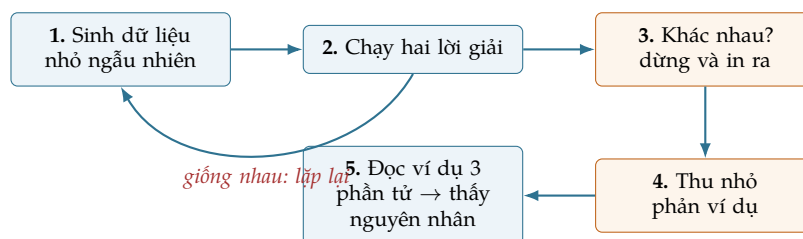
Đây là quy trình thay thế cho việc sửa mò, và nó gồm bốn phần.

*Một — bộ sinh dữ liệu.* Một chương trình nhỏ sinh dữ liệu ngẫu nhiên *hợp lệ* với kích thước *rất nhỏ*:  $n \leq 8$ , giá trị trong  $[-3, 3]$ . Kích thước nhỏ là điểm mấu chốt, vì nó vừa làm lỗi dễ hiểu vừa làm các trường hợp trùng lặp và biên xuất hiện thường xuyên thay vì hiếm.

*Hai — lời giải trọng tài.* Một lời giải vét cạn, chậm nhưng bạn tin chắc là đúng. Đây chính là lý do chương C/13 nói “luôn viết vét cạn trước”: nó không chỉ giúp bạn nghĩ, nó còn là công cụ kiểm thử.

*Ba — vòng lặp so sánh.* Chạy cả hai trên cùng dữ liệu, so kết quả, dừng lại ở lần đầu khác nhau và in dữ liệu đó ra. Vài dòng kịch bản, chạy vài nghìn ca trong vài giây.

*Bốn — thu nhỏ.* Khi đã có một dữ liệu sai, hãy tìm dữ liệu *nhỏ hơn* vẫn sai: thử bỏ từng phần tử, giảm từng giá trị, và giữ lại thay đổi nào vẫn giữ được lỗi. Việc này tự động hoá được bằng một vòng lặp đơn giản, và nó biến một phần ví dụ 8 phần tử thành 3 phần tử — ở kích thước đó bạn thường nhìn ra nguyên nhân ngay lập tức.



**Hình 1.1.** Vòng stress test. Điểm quan trọng nhất là bước 1 phải sinh dữ liệu *rất nhỏ* — điều này vừa làm phần ví dụ dễ đọc vừa làm các trường hợp biên xuất hiện liên tục thay vì hiếm. Bước 4 thường bị bỏ, nhưng nó là bước biến một phần ví dụ khó hiểu thành một phần ví dụ hiển nhiên.

Stress test không chỉ dùng để gỡ lỗi. Nó còn là cách kiểm chứng một *ý tưởng* trước khi tin vào nó: khi bạn nghĩ ra một tiêu chí tham lam (C/15) hoặc một công thức quy hoạch động, năm phút stress test cho câu trả lời chắc chắn hơn nhiều so với mười phút cố chứng minh trong đầu.

#### CẦN NHỚ

Khi bí trong lúc gỡ lỗi, thứ tự nên làm là: (i) kiểm dữ liệu biên bằng tay —  $n = 0, 1$ , mọi phần tử bằng nhau; (ii) viết stress test; (iii) thu nhỏ phản ví dụ; (iv) chỉ khi đó mới đọc code. Đảo ngược thứ tự này — đọc code trước — là cách phần lớn thời gian gỡ lỗi bị lãng phí.

### 1.4 Khi không có lời giải trọng tài

Đôi khi bài quá khó để viết vết cạn, hoặc kết quả không so sánh trực tiếp được (có nhiều đáp án đúng). Vẫn còn ba cách.

*Kiểm tính chất thay vì kiểm giá trị.* Nếu bài yêu cầu một cách sắp xếp thoả điều kiện, hãy viết một hàm *kiểm tra* kết quả có hợp lệ không thay vì so với đáp án. Đây là ý tưởng nền của kiểm thử dựa trên tính chất, và nó thường dễ hơn viết vết cạn nhiều.

*So hai cách cài đặt khác nhau.* Nếu bạn có hai thuật toán cho cùng bài — ví dụ một bản đệ quy và một bản lặp, hoặc một bản dùng cây phân đoạn và một bản dùng phân rã căn — so chúng với nhau. Hai bản có thể cùng sai vì dùng chung mô hình hoặc cùng một giả thiết. So chéo là bằng chứng bổ sung, không thay thế trọng tài độc lập và các bất biến.

*Kiểm bất biến trong lúc chạy.* Rải khắp định kiểm các bất biến ở những điểm then chốt, rồi chạy trên dữ liệu lớn. Cách này không cần trọng tài và bắt được lỗi ngay tại nơi cấu trúc dữ liệu bị hỏng.

### 1.5 Gỡ lỗi như một bài toán tìm kiếm

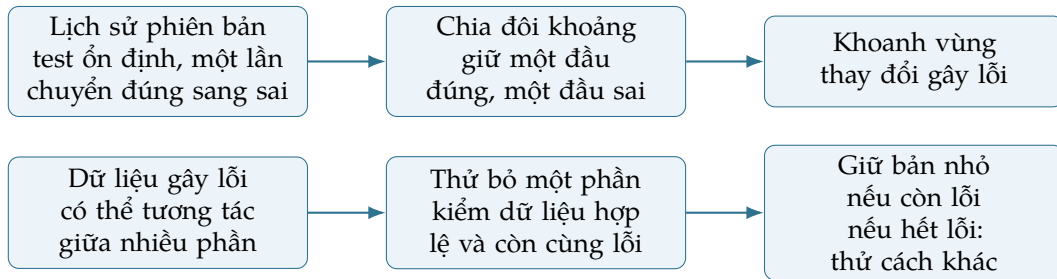
Khi đã có phản ví dụ nhỏ mà vẫn chưa thấy nguyên nhân, hãy coi việc gỡ lỗi là bài toán tìm kiếm — chọn phép thu hẹp theo cấu trúc của lỗi, thay vì mặc định mọi thứ đều đơn điệu.

Trên trục *không gian*: đặt các điểm kiểm tra ở giữa chương trình, xác nhận trạng thái tại đó đúng hay sai, rồi loại một nửa. Chỉ loại được một nửa nếu phép kiểm bảo đảm trạng thái lỗi không bị che hoặc sửa ở bước sau; nói chung trạng thái đúng/sai có thể xen kẽ.

Trên trục *thời gian*: nếu code trước đây chạy đúng, hãy tìm thay đổi đầu tiên làm hỏng bằng cách chia đôi lịch sử — đây chính là công cụ *git bisect*, và điều kiện dùng được của nó là tính đơn điệu, đúng như mọi tìm kiếm nhị phân khác.

Trên trục *dữ liệu*: đó chính là bước thu nhỏ phản ví dụ ở mục 3.

Hình 1.2 phân biệt phép chia đôi có bất biến với việc thu nhỏ dữ liệu phải kiểm lại sau mỗi lần thử.



**Hình 1.2.** Hai quy trình thu hẹp khác nhau. Trên: tìm ranh giới trong một lịch sử có giả thiết đơn điệu. Dưới: thu nhỏ phần ví dụ; không suy ra rằng một nửa đúng thì nửa còn lại chắc chắn chứa toàn bộ nguyên nhân.

Ba trục này dùng chung ý tưởng thu hẹp, nhưng không có cùng bảo đảm thuật toán, và nhận ra điều đó giúp bạn luôn có một nước đi tiếp theo thay vì bế tắc.

## 1.6 Ba việc riêng cho chương trình có ngẫu nhiên

Câu hỏi mở đầu thứ năm hỏi về việc gỡ lỗi khi chương trình có yếu tố ngẫu nhiên, và câu trả lời gồm ba việc rất cụ thể.

*Ghi lại hạt giống.* Mỗi lần chạy, in ra hạt giống đã dùng. Khi gặp lỗi, chạy lại với đúng hạt giống đó và bạn tái lập được. Không có bước này thì mọi lỗi phụ thuộc ngẫu nhiên đều gần như không sửa được.

*Tách phần ngẫu nhiên khỏi phần logic.* Nếu nguồn ngẫu nhiên được truyền vào như một tham số thay vì gọi trực tiếp trong hàm, bạn thay được nó bằng một dãy cố định khi kiểm thử.

*Kiểm bằng thống kê, không bằng một lần chạy.* Với thuật toán ngẫu nhiên, một lần chạy đúng không nói gì. Hãy chạy hàng nghìn lần và kiểm phân phối kết quả — ví dụ với một hàm trộn ngẫu nhiên, đếm tần suất mỗi hoán vị trên  $n = 4$  và kiểm chúng gần đều nhau.

## 1.7 Biến mỗi lỗi thành một phép kiểm có thể chạy lại

Một lần sửa chỉ có giá trị lâu dài nếu phần ví dụ trở thành ca kiểm thử hồi quy. Bảng 1.1 đưa ra ba lớp kiểm bổ sung nhau. Với một bài trả về nhiều lời giải hợp lệ, hãy so giá trị mục tiêu và dùng bộ kiểm chứng tính hợp lệ, không so nguyên văn hai danh sách kết quả.

**Bảng 1.1.** Ba lớp kiểm thử và giới hạn của bằng chứng chúng cung cấp.

| Lớp kiểm            | Ví dụ                                                  | Bắt được                          | Chưa bảo đảm                          |
|---------------------|--------------------------------------------------------|-----------------------------------|---------------------------------------|
| Ca biên có chủ đích | Mảng rỗng, toàn âm, giá trị trùng, không có nghiệm     | Sai khởi tạo và quy ước biên      | Phối hợp bất ngờ giữa nhiều điều kiện |
| So với vết cặn      | Duyệt mọi tập con ở $n \leq 12$ rồi so giá trị         | Sai logic của thuật toán nhanh    | Lỗi chỉ xuất hiện khi dữ liệu lớn     |
| Quan hệ biến đổi    | Sắp lại đầu vào không đổi MST; đảo đoạn không đổi tổng | Sai phụ thuộc thứ tự ngoài đặc tả | Đúng cho mọi dữ liệu                  |

Khi test sai, lưu cả đầu vào cụ thể, kết quả mong đợi, kết quả thực tế và seed; seed một mình chưa đủ nếu generator hay phiên bản thư viện đã đổi. Thu nhỏ nhưng giữ đúng đặc tả: bỏ một cạnh có thể làm đồ thị không còn liên thông, khiến test không còn thuộc miền bài toán. Sau khi



sửa, chạy lại ca gốc, ca đã thu nhỏ và bộ đối chiếu; đừng chỉ chạy đúng ví dụ nhỏ vừa dùng để sửa.

## 1.8 Gốc rễ: từ khẳng định của Turing tới thu nhỏ tự động

Ý tưởng đặt khẳng định vào chương trình để kiểm tra trạng thái xuất hiện rất sớm — Turing đã dùng nó năm 1949 trong bài viết về kiểm tra một chương trình lớn, như chương A/04 đã kể. Nhưng phải mất nhiều thập kỷ để nó thành thói quen phổ biến, và lý do là bối cảnh: khi máy tính còn đắt, việc để chương trình tự dừng vì một khẳng định bị coi là lãng phí thời gian máy.

Câu nói của Dijkstra — kiểm thử cho thấy sự hiện diện của lỗi, không cho thấy sự vắng mặt — thường bị dùng như một lời chê kiểm thử, nhưng ý ông tinh tế hơn: kiểm thử là một công cụ *phát hiện* rất tốt và một công cụ *bảo chứng* rất tệ, nên đừng nhầm hai vai trò. Toàn bộ mục 3 của chương này khai thác đúng vai trò thứ nhất.

Ba kỹ thuật hiện đại đáng nhắc vì chúng đều là hình thức hoá của những việc mà người giải bài vẫn làm bằng tay. *Kiểm thử ngẫu nhiên có hệ thống* bắt đầu được nghiên cứu nghiêm túc từ đầu thập niên 1990, khi người ta phát hiện rằng ném dữ liệu ngẫu nhiên vào các tiện ích hệ thống làm sập một tỉ lệ đáng ngạc nhiên trong số chúng — một kết quả gây sốc vì các phần mềm đó đã được dùng hàng ngày trong nhiều năm. *Kiểm thử dựa trên tính chất* ra đời cuối thập niên 1990 với ý tưởng: thay vì viết từng ca kiểm thử, hãy phát biểu một tính chất phải luôn đúng và để máy sinh dữ liệu tìm phản ví dụ — đúng mục 4 ở trên. Và *thu nhỏ tự động* được hình thức hoá cuối thập niên 1990 dưới tên gỡ lỗi delta: một quy trình thử các tập con và phần bù để tìm phản ví dụ 1-tối-tiểu (bỏ thêm một đơn vị thì không còn tái hiện lỗi), không bảo đảm nhỏ nhất toàn cục<sup>[1]</sup>, tức là bước 4 ở mục 3 nhưng có phân tích chi phí đàng hoàng.

Điểm chung của ba kỹ thuật: chúng đều chuyển công việc từ người sang máy, và cả ba đều dựa trên cùng một quan sát — *máy tìm phản ví dụ giỏi hơn người, còn người hiểu phản ví dụ giỏi hơn máy*. Quy trình ở mục 3 chỉ là cách phân công đúng theo quan sát đó.

### Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                                                   | Bài toán thúc ép lúc đó                                                 | Điều thay đổi sau đó                                                                                              |
|-------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 1949 — Turing                                         | Kiểm tra một chương trình dài mà chạy thử không xuể.                    | Khẳng định tại từng điểm; ý tưởng có từ rất sớm nhưng chỉ thành thói quen phổ biến khi thời gian máy trở nên rẻ.  |
| Thập niên 1970 — Dijkstra                             | “Khủng hoảng phần mềm”: lỗi vượt khả năng kiểm soát bằng chạy thử.      | Kiểm thử là công cụ phát hiện lỗi, không phải bảo chứng — phân vai đúng thay vì bỏ kiểm thử.                      |
| Đầu thập niên 1990 — kiểm thử bằng dữ liệu ngẫu nhiên | Ném dữ liệu ngẫu nhiên vào các tiện ích hệ thống làm sập nhiều bất ngờ. | Chứng minh rằng phần mềm dùng hàng ngày vẫn đầy lỗi ở dữ liệu bất thường; sinh ra cả một nhánh kiểm thử tự động.  |
| Cuối thập niên 1990 — gỡ lỗi delta                    | Phản ví dụ tìm được thường quá lớn để hiểu.                             | Thuật toán thử tập con và phần bù để thu nhỏ dữ liệu — chính là bước 4 của stress test, có phân tích đàng hoàng.  |
| Khoảng 2000 — kiểm thử dựa trên tính chất             | Viết từng ca kiểm thử vừa tốn công vừa bỏ sót.                          | Phát biểu tính chất phải luôn đúng, để máy sinh dữ liệu tìm phản ví dụ; cách làm khi không có lời giải trọng tài. |

## 1.9 Liên hệ ngang

## Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực                   | Ý tưởng tương ứng                                      | Giống ở đâu, khác ở đâu                                                                                                                                                                                           |
|----------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kiểm chứng phần cứng       | Mô phỏng ngẫu nhiên có ràng buộc, kiểm chứng hình thức | Cùng cấu trúc với stress test nhưng ở quy mô công nghiệp: sinh kích thích ngẫu nhiên hợp lệ và so với một mô hình tham chiếu — chính là “lời giải trọng tài” của mục 3.                                           |
| Hàng không                 | Bảng kiểm trước khi cất cánh                           | Danh mục lỗi ở mục 2 chính là một bảng kiểm. Bài học từ ngành hàng không: bảng kiểm hiệu quả không phải vì phi công không biết, mà vì nó chống lại việc <i>bỏ sót dưới áp lực</i> — đúng tình huống trong kỳ thi. |
| Y khoa                     | Chẩn đoán phân biệt, xét nghiệm loại trừ               | Gỡ lỗi bằng cách chia đôi vùng nghi ngờ giống hệt việc loại trừ dần các chẩn đoán. Cùng nguyên tắc: chọn phép thử nào loại được nhiều khả năng nhất, không phải phép thử dễ nhất.                                 |
| Khoa học thực nghiệm       | Tái lập kết quả, ghi sổ thí nghiệm                     | Việc ghi hạt giống ngẫu nhiên ở mục 6 chính là yêu cầu tái lập của khoa học. Một kết quả không tái lập được thì không kiểm chứng được — đúng với thí nghiệm và với lỗi phần mềm.                                  |
| Ước lượng trạng thái, SLAM | Chạy trên đoạn dữ liệu ngắn, kiểm bất biến của bộ lọc  | Áp dụng trực tiếp mục 3 và 4: dựng “ví dụ nhỏ nhất” là một đoạn vài giây thay vì cả chuỗi, và kiểm các bất biến (hiệp phương sai xác định dương, hướng trọng lực không trôi) bằng khẳng định trong lúc chạy.      |
| Thống kê                   | Kiểm định giả thuyết                                   | Việc kiểm thuật toán ngẫu nhiên bằng phân phối kết quả (mục 6) là một bài kiểm định thống kê. Cạm bẫy cũng giống: một lần quan sát không nói gì, và cần đủ số lần lặp để kết luận có nghĩa.                       |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai lý do. Thứ nhất, bạn đọc lại chính suy nghĩ của mình nên bạn thấy điều bạn *định* viết chứ không phải điều bạn *đã* viết — đây là hiện tượng tâm lý rất mạnh và không khắc phục được bằng cách cố gắng hơn. Thứ hai, đọc lại không tạo ra thông tin mới; bạn đang lặp lại cùng một biểu diễn, đúng trạng thái bí mà A/01 mô tả. Stress test thì tạo ra thông tin mới: một dữ liệu cụ thể mà chương trình sai, và đó là thứ thu hẹp không gian nguyên nhân.
- A2. Vì lời giải vét cạn thường chỉ tốn năm tới mười phút — nó không cần nhanh, không cần đẹp, chỉ cần đúng — trong khi việc sửa mò một lỗi không hiểu có thể tốn hàng giờ và không có gì bảo đảm kết thúc. Ngoài ra bạn thường đã viết vét cạn rồi, vì C/13 khuyến viết nó như bước đầu của quá trình nghĩ. Và nó còn dùng lại cho các bài sau: một bộ sinh test và một khung stress test viết một lần dùng mãi.
- A3. Vì số lượng thứ có thể sai tỉ lệ với kích thước dữ liệu. Trên mảng ba phần tử, bạn theo dõi được toàn bộ trạng thái bằng mắt và thường nhìn ra nguyên nhân trong một phút. Trên mảng ba nghìn phần tử, bạn không biết bắt đầu từ đâu, và các lỗi khác nhau trông giống hệt nhau ở đầu ra. Việc thu nhỏ tự động hoá được bằng một vòng lặp thử bỏ từng phần tử, nên chi phí gần như bằng không so với lợi ích.
- A4. Lỗi ở trường hợp *biên* và trường hợp *cấu trúc đặc biệt* — mảng rỗng, một phần tử, mọi phần tử bằng nhau, đồ thị không liên thông, giá trị lớn nhất gây tràn. Bộ sinh ngẫu nhiên thông thường hầu như không tạo ra chúng. Cách phòng có hai phần: sinh dữ liệu với *miền giá trị rất nhỏ* để ép trùng lặp và biên xuất hiện thường xuyên; và bổ sung một danh sách ca biên viết tay chạy trước mọi lần nộp.

- A5.** Ba việc. Ghi lại hạt giống mỗi lần chạy và in ra, để tái lập được đúng lần chạy hỏng. Truyền nguồn ngẫu nhiên vào như tham số thay vì gọi trực tiếp, để thay được bằng dãy cố định khi kiểm thử. Và kiểm bằng thống kê thay vì bằng một lần chạy — chạy hàng nghìn lần rồi kiểm phân phối, vì với thuật toán ngẫu nhiên thì một lần đúng không chứng minh gì.

### NÓI LẠI THẬT ĐƠN GIẢN

Khi chương trình sai mà bạn không biết vì sao, phản xạ tự nhiên là đọc lại code. Đó gần như luôn là cách chậm nhất, vì bạn sẽ đọc thấy cái mình *định* viết chứ không phải cái mình *đã* viết — giống như tự dò lỗi chính tả trong bài văn của chính mình.

Cách nhanh hơn nhiều là bắt máy tìm hộ. Bạn viết thêm hai chương trình rất ngắn: một cái sinh ra dữ liệu nhỏ ngẫu nhiên, và một cái giải bài theo cách ngu ngốc nhất nhưng chắc chắn đúng — thử hết mọi khả năng. Rồi cho cả hai chạy hàng nghìn lần và so đáp án. Khi chúng khác nhau, bạn có trong tay một ví dụ cụ thể mà chương trình sai.

Còn một bước nữa mà nhiều người bỏ, và nó là bước đáng giá nhất: hãy làm cho ví dụ đó *nhỏ hết mức*. Thử bỏ bớt một phần tử, nếu vẫn sai thì bỏ tiếp. Một ví dụ sai với ba phần tử thường tự nói ra nguyên nhân; cùng cái lỗi ấy trên ba nghìn phần tử thì không nói gì cả.

Cả quy trình này chỉ tốn mười phút để viết lần đầu, và bạn dùng lại nó cho mọi bài sau. So với việc ngồi sửa mò nửa tiếng và không chắc đã xong, đó là một trong những đánh đổi tốt nhất trong nghề.

Cuối cùng, gỡ lỗi là tìm kiếm bằng các phép kiểm có thể lặp lại. Dùng chia đôi khi có ranh giới đúng/sai; với dữ liệu, thử thu nhỏ và chỉ giữ thay đổi còn tái hiện cùng lỗi. Một phản ví dụ không thu nhỏ thêm được theo các phép thử hiện có chưa chắc nhỏ nhất toàn cục.

**Một câu:** *Đừng đọc lại code khi bí — hãy để máy tìm cho bạn một ví dụ sai nhỏ.*

**Chỗ không rút gọn được:** Lỗi phụ thuộc thời gian hoặc phụ thuộc môi trường (nhiều luồng, phân cứng) không tái lập được bằng stress test thông thường; đó là một lớp khó riêng.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                     | Trả lời                                                                                                                                          |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| ••• Vì sao không nên đọc lại code trước | Bạn đọc thấy điều mình định viết, và không tạo ra thông tin mới. Stress test tạo ra thông tin mới: một dữ liệu cụ thể sai.                       |
| ••• Bốn bước stress test                | Sinh dữ liệu <i>rất nhỏ</i> → chạy lời giải nhanh và vét cạn → dừng ở lần khác nhau đầu tiên → thu nhỏ phản ví dụ.                               |
| ••• Vì sao dữ liệu phải nhỏ             | Phản ví dụ dễ đọc, và các trường hợp trùng lặp/biên xuất hiện thường xuyên thay vì hiếm. Miền giá trị nhỏ cũng quan trọng như $n$ nhỏ.           |
| ••• Thứ tự khi gỡ lỗi                   | Kiểm ca biên bằng tay → stress test → thu nhỏ → mới đọc code. Đảo ngược thứ tự này là cách lãng phí thời gian phổ biến nhất.                     |
| ••• Gỡ lỗi là bài toán tìm kiếm         | Chia đôi trên ba trục: không gian (điểm kiểm tra giữa chương trình), thời gian (git bisect), dữ liệu (thu nhỏ phản ví dụ).                       |
| ••• Danh mục lỗi kinh điển              | Trần số; lệch một đơn vị; chưa khởi tạo; hàm so sánh không nhất quán; đệ quy quá sâu; chia số nguyên; phần dư âm; $n = 0, 1$ ; đọc dữ liệu chậm. |
| •• Khi không có lời giải trọng tài      | Kiểm <i>tính chất</i> thay vì giá trị; so hai cách cài đặt khác nhau; rài khẳng định kiểm bất biến trong lúc chạy.                               |
| •• Nơi nên đặt khẳng định               | Ở những chỗ bạn <i>chắc chắn</i> luôn đúng — vì đó là các giả định ngầm mà bạn sẽ không nghĩ tới khi gỡ lỗi.                                     |
| •• Ba việc với chương trình ngẫu nhiên  | Ghi và in hạt giống; truyền nguồn ngẫu nhiên như tham số; kiểm bằng phân phối trên nhiều lần chạy chứ không bằng một lần.                        |
| •• Quy ước đoạn                         | Chọn nửa mở $[l, r)$ hoặc đóng $[l, r]$ và giữ nhất quán cả bài; nửa mở ít gây lỗi hơn vì độ dài bằng $r - l$ .                                  |

| Thuật ngữ / câu hỏi                | Trả lời (tiếp)                                                                                                                 |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| ●● Stress test không chỉ để gỡ lỗi | Còn để <i>kiểm chứng ý tưởng</i> : năm phút stress test một tiêu chí tham lam chắc chắn hơn mười phút cổ chứng minh trong đầu. |
| ● Câu của Dijkstra, hiểu đúng      | Kiểm thử là công cụ phát hiện lỗi rất tốt và công cụ bảo chứng rất tệ — đừng nhầm hai vai trò, không phải dùng kiểm thử.       |

●●● phải thuộc ●● nên nắm ● biết là có

## I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

### 1. Viết bộ sinh test tốn bao lâu, và có đáng trong kỳ thi không?

| Khoảng năm phút nếu bạn đã có khung sẵn, và đáng trong hầu hết tình huống. Phép so sánh đúng là: bạn đang cân nhắc giữa năm phút chắc chắn có tiến triển và một khoảng thời gian không xác định để sửa mò. Ngoại lệ hợp lý: khi bạn còn dưới mười phút và lỗi rõ ràng là lỗi cài đặt nhỏ. Lời khuyên thực dụng: chuẩn bị sẵn khung stress test trong thư viện cá nhân (A/02) để mỗi lần chỉ phải viết bộ sinh.

### 2. Độ quy của tôi sập với $n$ lớn nhưng chạy tốt với $n$ nhỏ. Sửa thế nào?

| Đó gần như chắc chắn là tràn ngăn xếp. Ba lựa chọn: tăng giới hạn ngăn xếp nếu môi trường cho phép; chuyển sang bản lập với ngăn xếp tường minh (B/07); hoặc giảm độ sâu bằng cách đổi thứ tự duyệt — ví dụ với cây, duyệt con nặng sau cùng. Cách kiểm nhanh: chạy thử với dữ liệu hình đường thẳng (cây suy biến thành chuỗi), vì đó là trường hợp độ sâu lớn nhất và bộ sinh ngẫu nhiên hầu như không tạo ra.

### 3. Làm sao biết lỗi là tràn số?

| Ba dấu hiệu. Kết quả âm ở nơi phải dương; kết quả đúng với dữ liệu nhỏ và sai với dữ liệu lớn; hoặc kết quả sai lệch một lượng gần bằng lũy thừa của 2. Cách kiểm nhanh nhất là đổi tạm mọi kiểu số sang 128 bit hoặc sang số nguyên lớn và chạy lại — nếu đúng thì bạn đã xác định được nguyên nhân. Cách phòng là ước lượng giá trị lớn nhất *trước khi cài*, cùng lúc với việc ước lượng độ phức tạp (A/03).

### 4. Tôi nên đặt bao nhiêu khẳng định?

| Đủ để mỗi cấu trúc dữ liệu quan trọng được kiểm bất biến ít nhất một lần mỗi thao tác, nhưng không nhiều tới mức làm chậm chương trình đáng kể. Quy tắc thực dụng: đặt khẳng định ở ranh giới giữa các phần — đầu vào của một hàm, sau khi cập nhật một cấu trúc — và ở những giả định ngầm. Nhớ tắt chúng trong bản nộp nếu chúng tốn thời gian; nhưng đừng xóa, chỉ tắt.

### 5. Chương trình chạy đúng trên máy tôi, sai trên máy chấm. Nguyên nhân hay gặp?

| Bốn nguyên nhân theo thứ tự tần suất. Biến chưa khởi tạo — giá trị rác khác nhau giữa hai môi trường. Hành vi không xác định như tràn số có dấu hoặc truy cập ngoài mảng — có thể “chạy đúng” ở nơi này và sai ở nơi khác. Khác biệt về kích thước kiểu số hoặc về cách làm tròn số thực. Và giới hạn ngăn xếp khác nhau. Cả bốn đều bắt được bằng cách bật các tùy chọn kiểm tra của trình biên dịch khi chạy thử ở nhà.

### 6. Có cách nào kiểm thử một lời giải mà bài có nhiều đáp án đúng?

| Có, và đó là mục 4: viết một hàm *kiểm tra tính hợp lệ* thay vì so với đáp án mẫu. Nếu bài yêu cầu một cách sắp xếp thỏa điều kiện, hàm kiểm tra chỉ cần duyệt và xác nhận mọi điều kiện được thỏa. Nếu bài yêu cầu một lời giải tối ưu, hãy kiểm hai điều: kết quả hợp lệ, và giá trị của nó bằng giá trị mà lời giải vét cạn tìm được — so *giá trị* chứ không so cấu hình.

### 7. Trong công việc thật, quy trình này thay đổi thế nào?

| Cùng nguyên tắc, khác quy mô. Lời giải vét cạn trở thành một mô hình tham chiếu đơn giản hoặc một phiên bản cũ đã tin cậy. Bộ sinh dữ liệu trở thành sinh dữ liệu có ràng buộc theo miền nghiệp vụ. Bước thu nhỏ trở thành việc rút gọn một ca lỗi từ hệ thống thật thành một ca tái lập được. Và có

thêm hai thứ mà thi đấu không có: kiểm thử hồi quy — mỗi lỗi đã sửa trở thành một ca kiểm thử vĩnh viễn — và ghi nhật ký đủ để tái lập sự cố từ hiện trường.

### **Bài tập** (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                                       | Nguồn                   | Điều cần nhận ra                                                                                |
|---------------------------------------------------------------------------|-------------------------|-------------------------------------------------------------------------------------------------|
| ● Viết khung stress test dừng lại được cho thư viện cá nhân               | Bài tập công cụ         | Gồm bộ sinh, vòng so sánh, và hàm thu nhỏ; đây là bài tập có lợi ích lâu dài nhất chương.       |
| ● Viết lại tìm kiếm nhị phân và stress test nó                            | [2]                     | So với thư viện chuẩn trên hàng nghìn ca ngẫu nhiên, gồm mảng rỗng và mảng trùng lặp.           |
| ●● Dừng stress test để tìm phân ví dụ cho một tiêu chí tham lam sai       | C/15                    | Mục tiêu kép: luyện công cụ và luyện thói quen không tin cảm giác.                              |
| ●● Viết hàm thu nhỏ tự động cho phân ví dụ dạng mảng                      | Bài tập công cụ         | Thứ bỏ từng phần tử và giảm từng giá trị; giữ thay đổi nào vẫn còn lỗi.                         |
| ●● Cố tình tạo một lỗi tràn số rồi tìm nó bằng quy trình ở mục 5          | Bài tập luyện quy trình | Đi đúng thứ tự bốn bước; đo thời gian để thấy quy trình nhanh hơn sửa mò.                       |
| ●● Chuyển một thuật toán đệ quy sâu sang bản lặp                          | Bài tập cài đặt         | Duyệt sâu trên đồ thị $10^5$ đỉnh hình đường thẳng; kiểm cả hai bản cho cùng kết quả.           |
| ●●● Viết bộ kiểm tra tính hợp lệ cho một bài có nhiều đáp án đúng         | Bài tập kiểm thử        | Ví dụ: sắp xếp tô-pô, ghép đôi, tô màu; kiểm mọi ràng buộc của đề.                              |
| ●●● Kiểm một hàm trộn ngẫu nhiên bằng thống kê                            | Bài tập kiểm thử        | Đếm tần suất mọi hoán vị với $n = 4$ trên $10^6$ lần chạy; phát hiện lỗi lệch phân phối nếu có. |
| ●●● Dừng git bisect để tìm thay đổi làm hỏng một bài trong kho mã của bạn | Bài tập công cụ         | Trải nghiệm trực tiếp tìm kiếm nhị phân trên trục thời gian.                                    |
| ●●● Lập danh sách ca biên riêng cho ba dạng bài bạn hay làm               | Bài tập tự tổ chức      | Ví dụ: bài mảng, bài đồ thị, bài chuỗi; biến nó thành bảng kiểm trước khi nộp.                  |

### Kết chương và đọc tiếp

Chương này thay việc sửa mò bằng một quy trình. Ba điều đáng mang theo: thứ tự đúng khi gỡ lỗi — ca biên, stress test, thu nhỏ, rồi mới đọc code; ý thức rằng máy tìm phân ví dụ giỏi hơn người còn người hiểu phân ví dụ giỏi hơn máy, nên hãy phân công theo đúng thể mạnh; và việc chọn phép thu hẹp phù hợp trên ba trục không gian, thời gian và dữ liệu.

Chương tiếp theo mở rộng phạm vi: từ “chương trình chạy đúng” sang “chương trình chạy đủ nhanh trong một hệ thống thật”. Ở đó, các kết luận của phần A về độ phức tạp tiệm cận gặp thực tế của hằng số, bộ nhớ và tính bảo trì — và bạn sẽ thấy rằng thuật toán tốt nhất trên giấy thường không phải thuật toán được chọn (E/26).

# Nguồn

---

- [1] A. Zeller and R. Hildebrandt, "Simplifying and Isolating Failure-Inducing Input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [2] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [3] D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [4] J. L. Bentley and M. D. McIlroy, "Engineering a Sort Function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [5] D. R. Musser, "Introspective Sorting and Selection Algorithms," *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [6] J. Bentley, *Writing Efficient Programs*. Prentice-Hall, 1982.
- [7] F. McSherry, M. Isard, and D. G. Murray, "Scalability! But at What COST?" In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [8] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [9] A. Laaksonen. "CSES Problem Set." (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [10] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [11] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [12] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [13] J. Erickson, *Algorithms*. Independently published, 2019.

## Thuật toán trong hệ thống thật (Algorithms in Production)

**PHỤ THUỘC** A/03 cho khoảng cách giữa tiệm cận và hằng số; D/24 cho các mô hình chi phí khác; E/25 cho kỷ luật đo đạc.

### TÓM TẮT

Chương này nói về chỗ mà lý thuyết gặp thực tế, và về vì sao thuật toán tốt nhất trên giấy thường không phải thuật toán được chọn. Bốn chủ đề: hằng số và bộ nhớ có tiếng nói riêng ở kích thước thật; kỷ luật đo trước khi tối ưu, vì trực giác về nút cổ chai gần như luôn sai; các tiêu chí ngoài tốc độ — độ trễ đuôi, khả năng bảo trì, tính dự đoán được — vốn quyết định nhiều hơn trong hệ thống thật; và một khảo sát ngắn về nơi các thuật toán của quyển sách này đang thực sự chạy. Nếu chỉ nhớ một điều: trong hệ thống thật, câu hỏi không phải “thuật toán nào nhanh nhất” mà là “ràng buộc thật sự của tôi là gì” — và câu trả lời thường không phải số phép toán.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **hồ sơ hiệu năng (profiling)** — đo xem thời gian thực sự bị tiêu ở đâu trong chương trình, thay vì đoán.
- **độ trễ đuôi** — thời gian phản hồi ở các phân vị cao — 99% hoặc 99,9% — thay vì trung bình; trong dịch vụ trực tuyến, đây mới là con số người dùng cảm nhận.
- **thông lượng so với độ trễ** — số việc làm được mỗi giây so với thời gian hoàn thành một việc; tối ưu cái này thường làm hại cái kia.
- **ngưỡng hoà vốn** — kích thước dữ liệu mà tại đó một thuật toán bậc thấp hơn bắt đầu thắng thuật toán có hằng số nhỏ hơn.
- **tối ưu hoá vi mô** — cải thiện hằng số mà không đổi thuật toán: đổi cấu trúc dữ liệu, giảm cấp phát, cải thiện kiểu truy cập bộ nhớ.
- **tính dự đoán được** — khả năng đảm bảo thời gian chạy cho *mỗi* lần gọi, không chỉ trung bình — yêu cầu đặc trưng của hệ thời gian thực.

### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao trực giác về “chỗ nào chậm” trong một chương trình lại gần như luôn sai?
- Q2. Khi nào một thuật toán có độ phức tạp tệ hơn lại là lựa chọn đúng trong sản phẩm thật?
- Q3. Vì sao độ trễ trung bình lại là con số gây hiểu lầm trong dịch vụ trực tuyến?
- Q4. Cùng một thuật toán, vì sao đổi cách sắp xếp dữ liệu trong bộ nhớ có thể làm nó nhanh gấp nhiều lần?
- Q5. Trong hệ thống thời gian thực, vì sao một cấu trúc có chi phí khấu hao rất tốt lại có thể là lựa chọn sai?

Có một khoảng cách mà người luyện thuật toán thường không thấy cho tới khi đi làm: trong kỳ thi, tiêu chí duy nhất là “đúng và trong giới hạn thời gian”. Trong hệ thống thật, có ít nhất năm tiêu chí cạnh tranh nhau — tốc độ trung bình, tốc độ ở trường hợp xấu, bộ nhớ, khả năng bảo



trì, và khả năng dự đoán — và thuật toán thắng cuộc là thuật toán cân bằng chúng, không phải thuật toán tối ưu một chiều.

Điều này không làm mất giá trị của phần A tới phần D. Ngược lại: bậc tiệm cận vẫn là thứ quyết định khi dữ liệu lớn lên, và không có nó thì bạn không có cách nào so sánh các hướng đi. Nhưng nó là *điều kiện cần chứ không đủ*, và chương này nói về phần còn thiếu.

## 2.1 Đo trước khi tối ưu

Đây là nguyên tắc quan trọng nhất chương, và lý do nó cần được nhắc lại là vì ai cũng đồng ý với nó mà rất ít người làm.

Câu của Knuth về tối ưu hoá sớm[3] thường bị trích cụt. Ý đầy đủ có hai vế: phần lớn thời gian ta nên bỏ qua các cải thiện nhỏ, *nhưng* không được bỏ lỡ cơ hội ở phần mã thực sự quan trọng — và cách biết phần nào quan trọng là *đo*, không phải đoán.

Vì sao trực giác sai? Ba lý do. Thứ nhất, chương trình hiện đại có nhiều tầng — thư viện, cấp phát bộ nhớ, hệ điều hành — và chi phí thường nằm ở tầng bạn không viết. Thứ hai, chi phí truy cập bộ nhớ không nhìn thấy được trong mã nguồn: hai vòng lặp trông giống hệt nhau có thể chênh nhau năm lần vì kiểu truy cập. Thứ ba, chúng ta có xu hướng chú ý tới đoạn mã *phức tạp* chứ không phải đoạn mã *chạy nhiều lần*, mà hai thứ đó thường không trùng nhau.

Quy trình tối thiểu: đo tổng thời gian trước; chạy hồ sơ hiệu năng để tìm phần chiếm nhiều nhất; chỉ tối ưu phần đó; đo lại để xác nhận. Bước cuối hay bị bỏ, và nó là bước quan trọng nhất — rất nhiều “tối ưu” hoá ra không cải thiện gì, hoặc làm chậm đi vì một hiệu ứng phụ.

## 2.2 Hằng số, bộ nhớ và những gì tiệm cận không nói

Ba nhóm yếu tố quyết định hiệu năng thật mà ký hiệu  $O(\cdot)$  bỏ qua.

*Kiểu truy cập bộ nhớ.* Đây là yếu tố lớn nhất và bị đánh giá thấp nhất, đúng như hình phân cấp bộ nhớ ở chương A/03. Duyệt mảng tuần tự nhanh hơn nhiều lần so với duyệt danh sách liên kết cùng số phần tử; duyệt ma trận theo hàng nhanh hơn theo cột; cấu trúc cây cài bằng mảng nhanh hơn cài bằng con trỏ. Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó cũng là lý do “tối ưu vi mô” đầu tiên nên thử luôn là đổi bố cục dữ liệu chứ không phải đổi thuật toán.

*Cấp phát bộ nhớ.* Cấp phát động trong vòng lặp là một trong những nguyên nhân chậm phổ biến nhất và ít bị nghi ngờ nhất. Cấp phát trước, dùng lại vùng nhớ, và tránh tạo đối tượng tạm là những cải thiện thường cho hệ số lớn mà không đổi một dòng logic nào.

*Ngưỡng hoà vốn.* Mọi thuật toán “tốt hơn về tiệm cận” đều có một kích thước dưới đó nó thua. Các thư viện thật xử lý điều này bằng cách kết hợp: hàm sắp xếp chuẩn chuyển sang sắp xếp chèn khi đoạn nhỏ[4], và chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng[5]. Đây là câu trả lời cho câu hỏi mở đầu thứ hai ở dạng cụ thể nhất: thuật toán “tệ hơn” được chọn khi  $n$  nhỏ, và trong thực tế  $n$  thường nhỏ hơn ta tưởng.

### CẠM BẮY

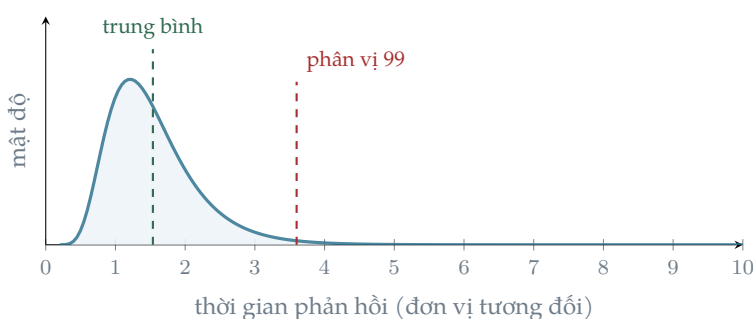
Một cạm bẫy đặc thù của người mới đi làm sau khi luyện thi đấu: chọn cấu trúc dữ liệu tinh vi cho một bài mà dữ liệu chỉ có vài trăm phần tử. Một mảng phẳng và một vòng lặp tuyến tính thường vừa nhanh hơn vừa dễ bảo trì hơn một cây cân bằng tự cài. Câu hỏi đầu



tiên trong hệ thống thật không phải “cấu trúc nào tối ưu” mà là “dữ liệu lớn tới đâu, và nó sẽ lớn tới đâu trong hai năm nữa”.

## 2.3 Những tiêu chí ngoài tốc độ

*Độ trễ đuôi.* Trong một dịch vụ trực tuyến, một yêu cầu của người dùng thường phải chờ hàng chục thành phần con trả lời. Nếu mỗi thành phần chậm bất thường trong 1% số lần, thì xác suất *ít nhất một* thành phần chậm là rất cao — và người dùng cảm nhận đúng cái chậm nhất. Đó là lý do các hệ thống lớn đo và tối ưu theo phân vị 99 hoặc 99,9 thay vì theo trung bình. Hệ quả cho việc chọn thuật toán: một thuật toán có trung bình tốt nhưng đuôi xấu — ví dụ bảng băm phải băm lại toàn bộ khi vượt ngưỡng tải, hay bộ thu gom rác chạy đột ngột — có thể tệ hơn một thuật toán chậm hơn nhưng đều đặn.



**Hình 2.1.** Minh họa bằng phân phối log-normal với  $\mu = 0.35, \sigma = 0.4$ , không phải số đo hệ thống thật. Trung bình xấp xỉ 1.537 và phân vị 99 xấp xỉ 3.599: phần lớn yêu cầu nhanh, một số ít rất chậm. Khi một thao tác của người dùng chạm tới hàng chục thành phần, họ cảm nhận cái chậm nhất — nên phân vị 99 mới là con số phản ánh trải nghiệm, và tính *đều đặn* của thuật toán trở nên quan trọng ngang tốc độ trung bình.

*Tính dự đoán được.* Trong hệ thời gian thực — điều khiển robot, xử lý âm thanh, hệ thống an toàn — yêu cầu là *mỗi* chu kỳ phải hoàn thành trước hạn. Ở đó, chi phí khấu hao đẹp không cứu được bạn: một thao tác đột ngột tốn  $O(n)$  làm lỡ hạn kỳ và hậu quả có thể là vật lý. Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó dẫn tới các lựa chọn khác hẳn: cấu trúc có đảm bảo trường hợp xấu nhất, cấp phát tĩnh thay vì động, và tránh các cơ chế chạy nền không kiểm soát được thời điểm.

*Khả năng bảo trì.* Một thuật toán mà chỉ một người trong nhóm hiểu là một rủi ro vận hành. Trong thực tế, chênh lệch 20% tốc độ hiếm khi biện minh được cho việc thay một lời giải rõ ràng bằng một lời giải tinh vi. Nguyên tắc thực dụng: dùng cấu trúc phức tạp khi nó cho cải thiện *bậc*, không phải khi nó cho cải thiện hằng số.

*Tính đúng dưới dữ liệu thù địch.* Đây là chỗ chương B/08 và C/20 chạm vào hệ thống thật: nếu dữ liệu vào đến từ người dùng bên ngoài, phân tích trung bình không còn đủ. Bảng băm không ngẫu nhiên hoá, biểu thức chính quy có thể bị đưa vào trạng thái mũ, và các thuật toán sắp xếp có trường hợp xấu đoán được — cả ba đều đã từng bị khai thác để làm quá tải dịch vụ.

## 2.4 Thuật toán của quyển sách này đang chạy ở đâu

Mục này là khảo sát ngắn, đọc để có bức tranh chứ không cần thuộc.

*Cơ sở dữ liệu* là nơi tập trung nhiều thuật toán nhất: B-cây cho chỉ mục (B/09), sắp xếp ngoài và trộn nhiều đường (D/24), nối bằng băm và nối bằng trộn, và một bộ tối ưu truy vấn về bản chất là bài toán tìm kiếm trên không gian kế hoạch thực thi.

*Trình biên dịch* dùng đồ thị ở khắp nơi (C/17): đồ thị luồng điều khiển, sắp thứ tự tô-pô để xếp lịch lệnh, tô màu đồ thị để cấp phát thanh ghi — một bài NP-khó được giải bằng heuristic trong mọi lần biên dịch.

*Mạng và hạ tầng* dùng Dijkstra và Bellman–Ford ở dạng phân tán cho định tuyến, băm nhất quán cho phân mảnh dữ liệu (B/08), và các cấu trúc xấp xỉ cho giám sát lưu lượng (D/24).

*Đồ hoạ và trò chơi* dùng cấu trúc phân cấp không gian cho dò tia và va chạm (D/22), A\* cho tìm đường, và rất nhiều tối ưu hằng số vì ràng buộc thời gian khung hình rất chặt.

*Robot và hệ thống nhận thức* dùng bài toán tối ưu trên đồ thị làm khung chính — đồ thị tư thế trong định vị và dựng bản đồ — cộng với cây  $k$ -d cho tìm lân cận, RANSAC cho loại ngoại lai (C/20), và A\* cho lập kế hoạch. Điểm đáng chú ý: ở đây ràng buộc thời gian thực và ràng buộc năng lượng thường quyết định nhiều hơn bậc tiệm cận, đúng như mục 3 nói.

*Học máy* dùng đại số tuyến tính và tối ưu liên tục làm lõi, nhưng các thuật toán rời rạc của quyển sách này vẫn có mặt ở khắp phần hạ tầng: lấy mẫu, chỉ mục vector cho tìm lân cận gần đúng, cấu trúc dữ liệu cho tải dữ liệu, và các bài chọn tập con có tính dưới mô-đun (C/15).

## 2.5 Một phiếu đo trước khi quyết định tối ưu

Với một pipeline xử lý khung hình, hãy đo riêng đọc dữ liệu, tiền xử lý, thuật toán lõi và xuất kết quả, rồi đo lại toàn pipeline. Giả sử phép tìm lân cận chiếm 30% tổng thời gian: tăng tốc phần này hai lần chỉ cho hệ số toàn cục

$$\frac{1}{(1 - 0.3) + 0.3/2} \approx 1.18.$$

Ngay cả làm phần đó nhanh vô hạn cũng chỉ đạt  $1/0.7 \approx 1.43$  lần. Đây là một phép tính từ tỉ phần thời gian giả định, không phải số đo benchmark.

**Bảng 2.1.** Phiếu đo để so sánh ba thay đổi thường gặp. Luôn dùng cùng tập đầu vào và kiểm kết quả trước khi so tốc độ.

| Thay đổi                           | Dự đoán cần kiểm                        | Đo thêm                              | Đánh đổi                                       |
|------------------------------------|-----------------------------------------|--------------------------------------|------------------------------------------------|
| Mảng phẳng thay nút cấp phát riêng | Giảm cấp phát và cache miss             | Bộ nhớ đỉnh, thời gian xây           | Chi phí chèn/xoá, di chuyển dữ liệu            |
| Tiền xử lý chỉ mục                 | Truy vấn rẻ hơn sau khi trả chi phí xây | Tổng thời gian theo số truy vấn $q$  | Chỉ mục mất hiệu lực khi dữ liệu đổi           |
| Xấp xỉ tìm lân cận                 | Đổi chất lượng lấy độ trễ thấp hơn      | Recall so với tìm chính xác, p95/p99 | Sai khác có thể ảnh hưởng bước tối ưu phía sau |

Bảng 2.1 buộc mỗi tối ưu có một dự đoán có thể bác bỏ. Ghi kích thước và phân phối dữ liệu, phiên bản mã, chế độ biên dịch, số luồng, phân cứng và số lần lặp. Tách phép đo cache nóng/lạnh; chạy xen kẽ bản cũ và mới để giảm sai lệch do nhiệt độ hoặc tải nền. Với robot, kiểm cả tỉ lệ trễ hạn trên toàn pipeline: p99 thấp không đồng nghĩa mọi chu kỳ đều kịp hạn.

## 2.6 Gốc rễ: từ “đừng tối ưu sớm” tới “đuôi mới là thứ quan trọng”

Câu chuyện về mối quan hệ giữa lý thuyết và thực hành trong ngành này đi qua vài giai đoạn khá rõ.

Thập niên 1970 là giai đoạn mà tối ưu hoá thủ công còn phổ biến và thường phản tác dụng. Knuth viết câu nổi tiếng về tối ưu hoá sớm trong bối cảnh đó[3], và Bentley viết cả một quyển sách về việc viết chương trình hiệu quả[6] với thông điệp cân bằng: có kỹ thuật thật, nhưng phải áp dụng ở đúng chỗ tìm được bằng đo đạc.

Thập niên 1990 mang tới một bài học khác từ việc kỹ thuật hoá các thư viện chuẩn. Bài viết của Bentley và McIlroy về hàm sắp xếp[4] là tài liệu đáng đọc vì nó cho thấy khoảng cách giữa thuật toán trong sách và hàm dùng được: phải xử lý dữ liệu đã sắp sẵn, dữ liệu có nhiều phần tử bằng nhau, dữ liệu thù địch, và phải chuyển thuật toán theo kích thước. Cùng thời kỳ, introsort[5] giải quyết vấn đề trường hợp xấu bằng cách kết hợp ba thuật toán.

Từ giữa thập niên 2000, khi dịch vụ trực tuyến quy mô lớn trở nên phổ biến, một mối quan tâm mới nổi lên: *độ trễ đuôi*. Nhận thức rằng trung bình là con số gây hiểu lầm khi một yêu cầu phụ thuộc vào hàng chục thành phần đã thay đổi cách các hệ thống lớn được thiết kế và đo lường — và nó cũng thay đổi tiêu chí chọn thuật toán, đưa tính đều đặn lên ngang hàng với tốc độ trung bình.

Cuối cùng, bài báo về chi phí của khả năng mở rộng[7] là một lời nhắc có tính hệ thống: nhiều hệ thống xử lý dữ liệu song song, khi đo công bằng, thua một chương trình đơn luồng viết cẩn thận. Thông điệp không phải “đừng song song hoá” mà là “hãy đo so với một mốc trung thực” — và mốc đó thường là một cài đặt đơn giản, tốt, chạy trên một máy.

### I Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                                      | Bài toán thúc ép lúc đó                                                                 | Điều thay đổi sau đó                                                                                    |
|------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 1974 — Knuth                             | Tối ưu hoá thủ công tràn lan làm mã khó đọc mà không nhanh hơn.                         | “Đừng tối ưu sớm” — nhưng về sau bị quên: phải đo để tìm phần thực sự quan trọng và tối ưu ở đó.        |
| 1982 — Bentley                           | Cần một cách có hệ thống để tăng tốc chương trình.                                      | Danh mục kỹ thuật kèm nguyên tắc: chỉ áp dụng ở nơi đo đạc chỉ ra.                                      |
| 1993 — Bentley & McIlroy                 | Hàm sắp xếp trong thư viện chuẩn hỏng trên dữ liệu thật.                                | Kỹ thuật hoá: xử lý dữ liệu đã sắp, nhiều phần tử bằng nhau, và chuyển thuật toán theo kích thước.      |
| 1997 — Musser                            | Quicksort có trường hợp xấu $O(n^2)$ khai thác được.                                    | Introsort: chuyển sang heapsort khi đệ quy quá sâu — kết hợp ba thuật toán để có cả tốc độ lẫn đảm bảo. |
| Giữa thập niên 2000 — dịch vụ quy mô lớn | Một yêu cầu phụ thuộc hàng chục thành phần; trung bình tốt mà người dùng vẫn thấy chậm. | Độ trễ đuôi trở thành tiêu chí chính; tính đều đặn được coi trọng ngang tốc độ trung bình.              |
| 2015 — McSherry và cộng sự               | Hệ thống phân tán được đánh giá bằng khả năng mở rộng chứ không bằng tốc độ tuyệt đối.  | So với mốc trung thực — một cài đặt đơn luồng tốt — nhiều hệ thống song song thua.                      |

## I 2.7 Liên hệ ngang

### I Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực                   | Ý tưởng tương ứng                                                                 | Giống ở đâu, khác ở đâu                                                                                                                                      |
|----------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kỹ thuật hệ thống          | Đo tải, phân vị độ trễ, ngân sách lỗi                                             | Cùng nguyên tắc “đo trước khi quyết”. Điểm khác so với thi đấu: mục tiêu không phải giá trị nhỏ nhất mà là <i>đạt một ngưỡng cam kết một cách ổn định</i> .  |
| Sản xuất công nghiệp       | Lý thuyết nút cổ chai: cải thiện chỗ không phải nút thắt thì không tăng sản lượng | Trùng khít với nguyên tắc hồ sơ hiệu năng ở mục 1. Bài học chung: tối ưu ngoài nút cổ chai vừa vô ích vừa che mắt vấn đề thật.                               |
| Kinh tế học                | Chi phí cơ hội, lợi ích cận biên giảm dần                                         | Mỗi giờ tối ưu hằng số là một giờ không dùng cho việc khác. Trong hệ thống thật, “đủ nhanh” là một trạng thái hợp lệ, khác hẳn tinh thần thi đấu.            |
| Hệ thời gian thực và robot | Ràng buộc hạn kỳ cứng, phân tích thời gian chạy xấu nhất                          | Ở đây tiêu chí là <i>trường hợp xấu nhất mỗi chu kỳ</i> , nên các cấu trúc có khấu hao đẹp bị loại. Đây là mô hình chi phí riêng mà D/24 đã nhắc.            |
| Kiến trúc máy tính         | Bộ nhớ đệm, dự đoán rẽ nhánh, xử lý theo lô                                       | Giải thích vì sao kiểu truy cập bộ nhớ quan trọng hơn số phép toán ở kích thước thật. Hiểu sơ bộ tăng này biến “tối ưu vi mô” từ mơ mẫm thành có định hướng. |
| Hàng không, y tế           | Thiết kế cho trường hợp xấu, không cho trường hợp trung bình                      | Cùng tinh thần với tính dự đoán được ở mục 3: trong hệ thống mà hậu quả nghiêm trọng, đảm bảo quan trọng hơn hiệu năng trung bình.                           |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì ba lý do. Chi phí thường nằm ở tầng bạn không viết — thư viện, cấp phát bộ nhớ, hệ điều hành. Chi phí truy cập bộ nhớ không nhìn thấy được trong mã nguồn, nên hai vòng lặp giống hệt nhau có thể chênh nhiều lần. Và chúng ta chú ý tới đoạn mã *phức tạp* thay vì đoạn mã *chạy nhiều lần*, mà hai thứ đó thường khác nhau. Đó là lý do quy trình bắt buộc phải là đo, tối ưu, rồi đo lại — và bước đo lại là bước hay bị bỏ nhất.
- A2. Ít nhất bốn tình huống. Khi  $n$  luôn nhỏ và nằm dưới ngưỡng hoà vốn — rất phổ biến hơn ta tưởng. Khi thuật toán “tệ hơn” có kiểu truy cập bộ nhớ tốt hơn và thắng trên máy thật. Khi nó đơn giản hơn nhiều và cả nhóm hiểu được, trong khi lợi ích chỉ là hằng số. Và khi nó có *đảm bảo trường hợp xấu nhất* tốt hơn, dù trung bình tệ hơn — điều quyết định trong hệ có hạn kỳ hoặc có dữ liệu thù địch.
- A3. Vì một yêu cầu của người dùng thường phụ thuộc vào nhiều thành phần con, và người dùng cảm nhận cái chậm nhất chứ không phải cái trung bình. Nếu mỗi thành phần chậm bất thường trong 1% số lần và một yêu cầu chậm tới vài chục thành phần thì xác suất gặp ít nhất một lần chậm là rất lớn. Vì vậy các hệ thống lớn đo theo phân vị cao, và tiêu chí chọn thuật toán chuyển từ “nhanh trung bình” sang “đều đặn”.
- A4. Vì đơn vị chi phí thật trên máy hiện đại là *lần chạm bộ nhớ chậm*, không phải phép toán. Dữ liệu nằm liên tiếp cho phép phần cứng nạp sẵn phần tiếp theo và tận dụng trọn vẹn mỗi dòng cache; dữ liệu nằm rải rác làm mỗi phần tử thành một lần chờ. Chênh lệch giữa hai trường hợp có thể tới hàng chục lần mà số phép toán không đổi — đúng như mô hình ở D/24 mô tả, chỉ ở quy mô cache thay vì đĩa.
- A5. Vì khấu hao chỉ đảm bảo *tổng* chi phí của một dãy thao tác, không đảm bảo từng thao tác. Trong hệ thời gian thực, một lần cấp phát lại toàn bộ hoặc một lần bấm lại bảng có thể tốn  $O(n)$  đúng vào chu kỳ đang có hạn kỳ, và hậu quả là lỡ hạn — trong điều khiển robot thì đó là hậu quả vật lý. Ở đó người ta chọn cấu trúc có đảm bảo trường hợp xấu nhất cho từng thao tác, chấp nhận hằng số lớn hơn và thông lượng thấp hơn.

### NÓI LẠI THẬT ĐƠN GIẢN

Trong kỳ thi, chỉ có một câu hỏi: chương trình có chạy đúng trong giới hạn thời gian không. Ngoài đời có thêm bốn câu nữa, và chúng thường quan trọng hơn.

Câu thứ nhất: chậm ở đâu? Câu trả lời gần như luôn khác chỗ bạn đoán. Người ta hay đi tối ưu đoạn mã trông rắc rối nhất, trong khi thời gian thật lại bị tiêu ở một vòng lặp tầm thường chạy hàng

triệu lần, hoặc ở việc cấp phát bộ nhớ mà bạn không hề viết ra. Cách duy nhất để biết là đo — và đo lại sau khi sửa, vì rất nhiều “cải tiến” hoá ra không cải tiến gì.

Câu thứ hai: dữ liệu thật lớn tới đâu? Nếu chỉ có vài trăm phần tử thì một vòng lặp đơn giản gần như luôn thắng một cấu trúc tinh vi, cả về tốc độ lẫn về việc người khác trong nhóm hiểu được nó.

Câu thứ ba: bạn quan tâm tới trung bình hay tới lần chậm nhất? Một dịch vụ có thời gian phản hồi trung bình rất đẹp vẫn có thể làm người dùng bức, vì mỗi thao tác của họ đọng tới hàng chục thành phần và họ cảm nhận cái chậm nhất. Còn trong điều khiển robot thì chỉ có lần chậm nhất là quan trọng: trung bình đẹp mà một chu kỳ lỡ hạn thì hỏng việc.

Câu thứ tư: ai sẽ sửa cái này sau một năm? Một lời giải nhanh hơn 20% nhưng chỉ một người hiểu thường là lựa chọn tệ hơn một lời giải rõ ràng.

Cuối cùng, một điều rất thực dụng mà lý thuyết không nói: trên máy hiện đại, việc dữ liệu nằm liền nhau trong bộ nhớ hay nằm rải rác có thể tạo chênh lệch nhiều lần, dù số phép tính không đổi. Vì vậy khi cần tăng tốc, thứ đầu tiên nên thử thường không phải đổi thuật toán mà là đổi cách xếp dữ liệu.

**Một câu:** Trong hệ thống thật, câu hỏi không phải “thuật toán nào nhanh nhất” mà “ràng buộc thật sự của tôi là gì”.

**Chỗ không rút gọn được:** Việc cân bằng giữa tốc độ, độ phức tạp mã và khả năng bảo trì không có công thức; nó là một quyết định kỹ thuật phụ thuộc bối cảnh và con người.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                              | Trả lời                                                                                                                                              |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| ●●● Nguyên tắc số một                            | Đo trước khi tối ưu, và <i>đo lại</i> sau khi tối ưu. Trực giác về nút cổ chai gần như luôn sai.                                                     |
| ●●● Ba lý do trực giác sai                       | Chi phí nằm ở tầng bạn không viết; chi phí bộ nhớ không thấy trong mã nguồn; ta chú ý mã <i>phức tạp</i> thay vì mã <i>chạy nhiều</i> .              |
| ●●● Yếu tố hiệu năng lớn nhất mà tiệm cận bỏ qua | Kiểu truy cập bộ nhớ. Đảo bố cục dữ liệu thường cho hệ số lớn hơn đối thuật toán, ở kích thước thật.                                                 |
| ●●● Khi nào chọn thuật toán “tệ hơn”             | $n$ luôn nhỏ; kiểu truy cập bộ nhớ tốt hơn; đơn giản hơn nhiều mà lợi ích chỉ là hằng số; hoặc có đảm bảo xấu nhất tốt hơn.                          |
| ●●● Độ trễ đuôi                                  | Người dùng cảm nhận thành phần chậm nhất, không phải trung bình; hệ thống lớn đo phân vị 99 hoặc 99,9. Tính đều đặn ngang hàng tốc độ.               |
| ●●● Hệ thời gian thực                            | Cần đảm bảo cho <i>mỗi</i> thao tác; khấu hao không đủ. Ưu tiên cấu trúc cố cận trên xấp xỉ, cấp phát tĩnh, tránh cơ chế chạy nền.                   |
| ●● Ngưỡng hoà vốn                                | Mọi thuật toán tốt hơn về tiệm cận đều thua ở kích thước nhỏ; thư viện thật giải quyết bằng cách kết hợp nhiều thuật toán theo kích thước và độ sâu. |
| ●● Cấp phát bộ nhớ                               | Cấp phát trong vòng lặp là nguyên nhân chậm phổ biến và ít bị nghi ngờ; cấp phát trước và dùng lại vùng nhớ thường cho cải thiện lớn.                |
| ●● Dữ liệu thù địch                              | Khi dữ liệu đến từ bên ngoài, phân tích trung bình không đủ: bảng băm phải ngẫu nhiên hoá, và các trường hợp xấu đoán được đều là lỗ hổng.           |
| ●● Khả năng bảo trì                              | Dùng cấu trúc phức tạp khi nó cho cải thiện <i>bậc</i> , không phải khi cho cải thiện hằng số.                                                       |
| ●● Ý đầy đủ của Knuth                            | Bỏ qua cải thiện nhỏ ở phần lớn mã, nhưng <i>không</i> bỏ lỡ ở phần thực sự quan trọng — và phải đo để biết phần nào.                                |
| ● Bài học COST                                   | So sánh với một mốc trung thực — một cài đặt đơn luồng tốt — trước khi kết luận rằng hệ thống song song nhanh hơn.                                   |

●●● phải thuộc    ●● nên nắm    ● biết là có

### Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

## 1. Tôi nên bắt đầu tối ưu từ đâu khi chương trình chậm?

| Theo thứ tự này. Một, đo để biết phần nào chiếm thời gian — đừng bỏ qua bước này dù bạn “chắc chắn” biết. Hai, kiểm tra bậc tiệm cận của phần đó: nếu sai bậc thì mọi tối ưu vi mô đều vô ích. Ba, nếu bậc đúng, xem kiểu truy cập bộ nhớ và cấp phát — hai thứ này thường cho hệ số lớn nhất. Bốn, chỉ sau đó mới tới các tối ưu vi mô khác. Và luôn đo lại sau mỗi thay đổi.

## 2. Làm sao biết mình đang bị chặn bởi tính toán hay bởi bộ nhớ?

| Hai thí nghiệm rẻ đã nói ở D/24: giảm kích thước dữ liệu xuống mức vừa bộ nhớ đệm và xem thời gian trên mỗi phần tử có giảm mạnh không; và đổi thứ tự truy cập mà giữ nguyên số phép toán. Nếu thời gian đổi nhiều lần trong thí nghiệm thứ hai thì bạn đang bị chặn bởi bộ nhớ, và việc cần làm là đổi bố cục dữ liệu chứ không phải giảm số phép tính.

## 3. Trong công việc, tôi có bao giờ cần tới các thuật toán ở phần C và D không?

| Có, nhưng không đều. Những thứ dùng gần như hằng ngày: sắp xếp và tìm kiếm, bảng băm, đồ thị cơ bản, quy hoạch động ở dạng đơn giản, nhị phân trên đáp án. Những thứ dùng vài lần trong sự nghiệp nhưng khi cần thì rất đáng: luồng cực đại cho các bài phân công, cấu trúc chuỗi cho tìm kiếm văn bản, nhận diện NP-khó để ngừng đúng lúc. Giá trị lớn nhất của phần C và D không phải là bạn cài chúng thường xuyên, mà là bạn *nhận ra* khi một bài trong công việc thuộc về một trong các mẫu đó.

## 4. Nên tự cài hay dùng thư viện?

| Mặc định là dùng thư viện: nó đã được kỹ thuật hoá, kiểm thử, và người khác đọc được. Tự cài khi có lý do cụ thể: thư viện thiếu thao tác bạn cần (tách/ghép cây, truy vấn thứ hạng), hằng số của nó là nút cổ chai đã đo được, hoặc bạn cần đảm bảo trường hợp xấu nhất mà thư viện không cho. Trong cả ba trường hợp, hãy stress test bản tự cài với thư viện làm trọng tài (E/25).

## 5. “Đủ nhanh” nghĩa là gì trong thực tế?

| Nghĩa là đạt một ngưỡng đã thoả thuận, ổn định, với biên an toàn cho tăng trưởng. Cách làm đúng là xác định ngưỡng *trước*: thời gian phản hồi mục tiêu ở phân vị 99, hoặc thời gian xử lý mỗi khung hình, hoặc dung lượng bộ nhớ tối đa. Không có ngưỡng thì việc tối ưu không có điểm dừng, và mọi giờ bỏ vào đó là một giờ không dùng cho việc khác — đây là nơi tinh thần kỹ thuật khác hẳn tinh thần thi đấu.

## 6. Người luyện thi đấu chuyển sang làm sản phẩm hay mắc lỗi gì?

| Ba lỗi. Chọn cấu trúc tinh vi cho dữ liệu nhỏ, làm mã khó bảo trì mà không nhanh hơn. Tối ưu hằng số trước khi kiểm tra bậc, hoặc ngược lại, đổi thuật toán khi nút cổ chai nằm ở nơi khác hoàn toàn. Và bỏ qua các tiêu chí ngoài tốc độ — đặc biệt là tính đều đặn và khả năng đọc hiểu. Điểm tốt mà họ mang sang: kỷ luật ước lượng chi phí trước khi cài, và thói quen kiểm thử bằng dữ liệu biên.

## 7. Có cách nào giữ được kỹ năng thuật toán khi công việc ít dùng tới?

| Ba cách rẻ. Duy trì một nhịp luyện nhỏ nhưng đều — một bài mỗi tuần đủ để giữ phản xạ (E/28). Chủ động tìm chỗ áp dụng trong công việc: rất nhiều bài “tối ưu lịch”, “chọn tập đại diện”, “tìm phụ thuộc vòng” là bài trong quyển sách này khoác áo nghiệp vụ. Và đọc mã nguồn của các thư viện bạn dùng — đó là nơi lý thuyết và kỹ thuật gặp nhau, và là tài liệu học tốt nhất về nội dung của chương này.

## Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                              | Nguồn               | Điều cần nhận ra                                                       |
|------------------------------------------------------------------|---------------------|------------------------------------------------------------------------|
| ● Đo thời gian duyệt mảng theo hàng và theo cột trên ma trận lớn | Bài tập thực nghiệm | Cùng số phép toán, khác kiểu truy cập; ghi lại tỉ số trên máy của bạn. |
| ● Chạy hồ sơ hiệu năng cho một chương trình bạn đã viết          | Bài tập công cụ     | So kết quả với dự đoán của chính bạn trước khi đo — thường rất khác.   |



| Bài                                                                                                     | Nguồn               | Điều cần nhận ra (tiếp)                                                                            |
|---------------------------------------------------------------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------|
| •• Tìm ngưỡng hoà vốn giữa sắp xếp chèn và sắp xếp trộn                                                 | Bài tập thực nghiệm | Vẽ đường thời gian theo $n$ ; so con số của bạn với ngưỡng mà thư viện chuẩn dùng.                 |
| •• So sánh danh sách liên kết và mảng động cho cùng khối lượng công việc                                | Bài tập thực nghiệm | Đo chèn, duyệt, xoá; giải thích kết quả bằng phân cấp bộ nhớ chứ không bằng độ phức tạp.           |
| •• Loại bỏ cấp phát trong vòng lặp nóng và đo lại                                                       | Bài tập tối ưu      | Cấp phát trước và dùng lại vùng nhớ; đây thường là tối ưu có tỉ lệ lợi ích trên công sức cao nhất. |
| •• Đo phân vị 50, 99 và 99,9 của thời gian phản hồi thay vì trung bình                                  | Bài tập đo lường    | Dùng một dịch vụ nhỏ tự viết; quan sát khoảng cách giữa trung bình và đuôi.                        |
| ••• Dựng dữ liệu thù địch cho một cấu trúc băm không ngẫu nhiên hoá                                     | B/08                | Đo mức suy giảm hiệu năng, rồi sửa bằng hạt giống ngẫu nhiên và đo lại.                            |
| ••• So một chương trình đơn luồng viết tốt với một phiên bản song song                                  | [7]                 | Chọn một bài xử lý đồ thị vừa phải; báo cáo cả thời gian lẫn số lỗi dùng.                          |
| ••• Thay một cấu trúc dựa trên con trỏ bằng mảng phẳng trong một chương trình có sẵn                    | Bài tập tối ưu      | Đo trước và sau; ghi lại cả thời gian lẫn mức tiêu thụ bộ nhớ.                                     |
| ••• Chọn một thuật toán trong quyển sách này và tìm nơi nó chạy trong một phần mềm mã nguồn mở bạn dùng | Bài tập đọc mã      | Đọc cách nó được kỹ thuật hoá so với bản trong sách; đây là bài tập học được nhiều nhất chương.    |

## Kết chương và đọc tiếp

Chương này nối phần lý thuyết với thực tế bằng một thay đổi trong câu hỏi: không phải “thuật toán nào nhanh nhất” mà “ràng buộc thật sự của tôi là gì”. Ba điều đáng mang theo: đo trước và đo lại sau khi tối ưu; kiểu truy cập bộ nhớ thường quyết định nhiều hơn số phép toán ở kích thước thật; và các tiêu chí ngoài tốc độ — đuôi độ trễ, tính dự đoán được, khả năng bảo trì — thường là thứ chọn ra thuật toán được dùng.

Hai chương cuối quay lại với người học. Chương [E/27](#) nói về chiến thuật khi có giới hạn thời gian và có người quan sát — phỏng vấn và thi đấu — nơi kỹ năng quan trọng nhất không phải kiến thức mà là quản lý thời gian và nói ra suy nghĩ. Sau đó, chương [E/28](#) biến toàn bộ quyển sách thành một lịch luyện cụ thể kèm ngân hàng bài tập.



## Nguồn

---

- [1] A. Zeller and R. Hildebrandt, “Simplifying and Isolating Failure-Inducing Input,” *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [2] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [3] D. E. Knuth, “Structured Programming with go to Statements,” *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [4] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [5] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [6] J. Bentley, *Writing Efficient Programs*. Prentice-Hall, 1982.
- [7] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [8] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [9] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [10] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [11] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [12] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [13] J. Erickson, *Algorithms*. Independently published, 2019.

## Phỏng vấn và thi đấu (Interviews and Contests)

**PHỤ THUỘC** A/01 cho quy trình bốn bước — ở đây nó được nén lại dưới sức ép thời gian; A/02 cho từng tự giám sát; E/25 cho bảng kiểm trước khi nộp.

### TÓM TẮT

Hai bối cảnh trong chương này — phỏng vấn và thi đấu — khác nhau ở tiêu chí chấm nhưng giống nhau ở điều quyết định kết quả: không phải lượng kiến thức mà là *quản lý thời gian và quản lý bản thân dưới sức ép*. Chương đưa ra một quy trình bảy bước cho buổi phỏng vấn, trong đó bước quan trọng nhất là nói ra suy nghĩ, vì người phỏng vấn chấm quá trình chứ không chỉ chấm kết quả. Với thi đấu, chương nói về đọc toàn bộ đề trước, phân bổ thời gian, quy tắc bỏ bài, và cách xử lý khi bị sai liên tiếp. Nếu chỉ nhớ một điều: dưới sức ép, thứ khiến bạn mất điểm nhiều nhất không phải bài khó mà là hai lỗi rất tránh được — hiểu sai đề, và bám vào một hướng sai quá lâu.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **nói ra suy nghĩ** — diễn đạt thành lời quá trình suy nghĩ trong lúc giải, để người nghe theo được đường đi của bạn.
- **thi ảo** — tự tổ chức một kỳ thi từ đề cũ, đúng thời lượng và điều kiện thật, để luyện chiến thuật chứ không chỉ luyện kiến thức.
- **phạt thời gian** — ở nhiều thể thức thi, mỗi lần nộp sai cộng thêm một khoảng vào tổng thời gian — điều này thay đổi hẳn chiến thuật nộp bài.
- **chấm từng phần** — thể thức cho điểm theo số test qua được hoặc theo nhóm test; khuyến khích nộp lời giải chậm hơn để lấy điểm bộ phận.
- **tilt** — trạng thái mất bình tĩnh sau một thất bại, dẫn tới các quyết định tệ tiếp theo — thuật ngữ mượn từ cờ và thể thao điện tử.
- **bảng kiểm trước khi nộp** — danh sách ngắn các ca biên và lỗi kinh điển cần rà lại, dùng để chống bỏ sót dưới áp lực (E/25).

### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao người phỏng vấn thường quan tâm tới quá trình suy nghĩ hơn là lời giải cuối cùng?
- Q2. Trong một kỳ thi ba tiếng với sáu bài, vì sao đọc hết cả sáu đề trước lại quan trọng hơn bắt tay vào bài đầu tiên ngay?
- Q3. Bạn đã bỏ 40 phút cho một bài và vẫn chưa ra. Quyết định đúng là gì, và làm sao biết được điều đó *trước khi hết giờ*?
- Q4. Vì sao im lặng suy nghĩ trong phỏng vấn lại là một sai lầm, kể cả khi bạn đang nghĩ tốt?
- Q5. Sau một lần nộp sai, vì sao xác suất lần nộp tiếp theo cũng sai lại cao hơn bình thường?

Hai bối cảnh trong chương này có một điểm chung mà người luyện thường không chuẩn bị: chúng không đo kiến thức thuần túy. Một kỳ thi ba tiếng đo khả năng *chọn bài, phân bổ thời gian và giữ bình tĩnh* nhiều không kém khả năng nghĩ ra thuật toán. Một buổi phỏng vấn 45 phút đo khả năng *giao tiếp và làm việc dưới quan sát* nhiều không kém năng lực kỹ thuật.

Điều đó có nghĩa là hai bối cảnh này luyện được riêng, và phần lớn người học bỏ qua việc luyện đó. Họ giải hàng trăm bài nhưng chưa bao giờ tự tổ chức một kỳ thi ảo đúng thời lượng, chưa bao giờ tập nói ra suy nghĩ trong lúc giải. Rồi đến ngày thật, họ mất điểm ở đúng những chỗ có thể luyện được.

### 3.1 Phỏng vấn: quy trình bảy bước

Một buổi phỏng vấn thuật toán điển hình kéo dài 40–60 phút cho một hoặc hai bài. Quy trình dưới đây nén bốn bước của Pólya (A/01) vào khung thời gian đó, và nó cũng chính là thứ người phỏng vấn mong thấy.

**Bước 1 — làm rõ đề (2–4 phút).** Hỏi về kích thước dữ liệu, kiểu dữ liệu, trường hợp biên, điều gì được đảm bảo. Đây là bước bị bỏ nhiều nhất và cũng là bước bị chấm điểm âm nặng nhất khi bỏ, vì trong công việc thật, viết code trước khi hiểu yêu cầu là một lỗi nghiêm trọng.

**Bước 2 — ví dụ (2 phút).** Tự viết một ví dụ nhỏ và giải nó bằng tay, xác nhận với người phỏng vấn rằng bạn hiểu đúng. Bước này bắt được phần lớn hiểu lầm và tốn rất ít thời gian.

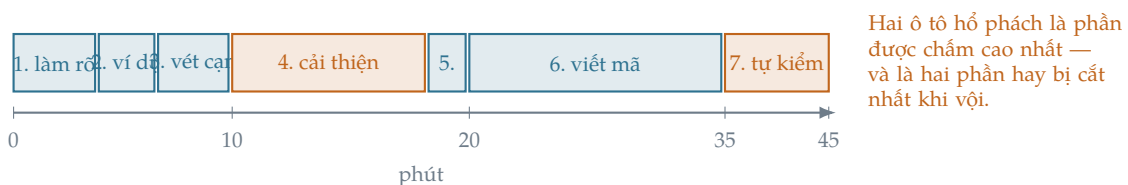
**Bước 3 — lời giải vét cạn (2–3 phút).** Nói ra cách giải chậm nhưng đúng, kèm độ phức tạp của nó. Đừng ngại: điều này cho thấy bạn hiểu bài, và nó tạo điểm xuất phát để cải thiện — đúng lập luận ở C/13.

**Bước 4 — cải thiện (5–10 phút).** Hỏi to câu “vét cạn này lãng phí ở đâu”, rồi đi theo cây quyết định ở hình của chương A/01. Đây là phần mà người phỏng vấn quan tâm nhất, và cũng là phần bạn nên nói nhiều nhất.

**Bước 5 — thống nhất trước khi viết (1 phút).** Tóm tắt lời giải trong ba câu và hỏi “tôi bắt đầu viết nhé”. Nếu hướng sai, người phỏng vấn thường gợi ý ở đúng lúc này — và bạn tiết kiệm được mười lăm phút.

**Bước 6 — viết mã (10–15 phút).** Viết sạch, đặt tên rõ, tách hàm khi hợp lý. Nói ngắn gọn khi làm điều gì đó không hiển nhiên.

**Bước 7 — tự kiểm (3–5 phút).** Chạy tay ví dụ ở bước 2 qua code, rồi rà bảng kiểm ca biên: mảng rỗng, một phần tử, giá trị trùng, tràn số. Chủ động làm bước này thay vì chờ được hỏi — nó tạo ấn tượng mạnh vì rất nhiều ứng viên bỏ qua.



**Hình 3.1.** Phân bố thời gian gợi ý cho một buổi phỏng vấn 45 phút. Điều đáng chú ý là bước 5 — thống nhất hướng trước khi viết — chỉ tốn một phút nhưng ngăn được kịch bản tốn kém nhất: viết mười lăm phút theo hướng sai. Bước 7 nên chủ động làm, không đợi được hỏi.

#### CẦN NHỚ

Điều được chấm không phải “có ra lời giải tối ưu không” mà là “người này có phải người tôi muốn cùng gỡ một bài khó lúc hai giờ sáng không”. Bốn thứ tạo nên ấn tượng đó: hỏi làm rõ trước khi làm, nói ra suy nghĩ, phản ứng tốt với gợi ý, và tự kiểm tra mà không đợi nhắc. Cả bốn đều luyện được, và cả bốn đều không liên quan tới việc bạn thuộc bao nhiêu thuật toán.

### Ba lỗi giao tiếp phổ biến

*Im lặng suy nghĩ.* Đây là câu trả lời cho câu hỏi mở đầu thứ tư: người phỏng vấn không đọc được suy nghĩ của bạn, nên với họ, ba phút im lặng trông giống hết ba phút bế tắc. Tệ hơn, họ không có cơ hội gợi ý đúng chỗ. Nếu cần nghĩ yên, hãy nói trước: “cho tôi một phút suy nghĩ về cấu trúc dữ liệu”.

*Nhảy vào viết code quá sớm.* Viết trước khi thống nhất hướng là cách nhanh nhất để mất mười lăm phút và phải xoá làm lại.

*Phỏng thủ khi được góp ý.* Gợi ý của người phỏng vấn gần như luôn là gợi ý thật, không phải bẫy. Cách phản ứng tốt là dừng lại, hỏi lại cho rõ, và đi theo hướng đó.

## 3.2 Thi đấu: chiến thuật quan trọng ngang kiến thức

Trong một kỳ thi có nhiều bài và giới hạn thời gian, ba quyết định chiến thuật ảnh hưởng tới kết quả nhiều hơn hầu hết kiến thức kỹ thuật.

*Quyết định 1 — đọc hết đề trước.* Dành 5–10 phút đầu đọc toàn bộ và ước lượng độ khó từng bài. Lý do rất thực dụng: thứ tự đề *không* luôn theo độ khó, và mỗi phút bạn dùng cho một bài khó trong khi còn một bài dễ chưa đọc là một phút bị lãng phí. Đây là câu trả lời cho câu hỏi mở đầu thứ hai. Nếu bảng điểm hiển thị số người giải được từng bài, đó là nguồn thông tin về độ khó chính xác hơn cảm giác của bạn.

*Quyết định 2 — đặt mốc thời gian và tuân thủ.* Trước khi bắt đầu một bài, tự đặt một hạn: “nếu 25 phút nữa chưa có hướng rõ ràng, tôi chuyển bài”. Không có mốc thì bạn sẽ phát hiện mình chọn sai bài lúc chỉ còn mười phút — đúng mô hình phân bố thời gian mà Schoenfeld quan sát được ở A/01. Có mốc thì bạn mất tối đa 25 phút cho một quyết định sai.

*Quyết định 3 — biết khi nào bỏ và khi nào quay lại.* Bỏ bài không phải thất bại. Rất thường xuyên, ý tưởng cho bài đã bỏ xuất hiện trong lúc bạn làm bài khác — hiện tượng “áp ú” ở A/01. Quy tắc thực dụng: khi bỏ, hãy ghi lại một dòng về chỗ bạn đang kẹt, để khi quay lại không phải nạp lại từ đầu.

### Nộp bài và xử lý sai

Thẻ thức chấm quyết định chiến thuật nộp. Nếu có *phạt thời gian* cho mỗi lần nộp sai, hãy dành thêm hai phút rà bảng kiểm trước khi nộp — rẻ hơn nhiều so với tiền phạt. Nếu có *chấm từng phần*, hãy nộp lời giải chậm hơn để lấy điểm bộ phận trước khi cố tối ưu; nhiều người mất điểm vì theo đuổi lời giải đầy đủ tới hết giờ.

Sau một lần nộp sai, có một hiệu ứng tâm lý cần biết, và nó là câu trả lời cho câu hỏi mở đầu thứ năm: xác suất lần nộp tiếp theo cũng sai cao hơn bình thường. Lý do là bạn thường sửa *triệu chứng* chứ không sửa nguyên nhân, và bạn làm việc đó dưới áp lực với sự chú ý bị thu hẹp. Cách chống là bắt buộc mình dừng lại 60 giây và trả lời hai câu trước khi sửa: *ca dữ liệu nào có thể làm sai*, và *giả định nào của tôi có thể không đúng*. Nếu không trả lời được, hãy viết stress test thay vì đoán tiếp (E/25).

## 3.3 Luyện hai kỹ năng này thế nào

Cả hai bối cảnh đều luyện được, và cách luyện rất cụ thể.

*Cho phỏng vấn:* giải bài với việc *nói to* toàn bộ suy nghĩ, kể cả khi ngồi một mình — ban đầu rất gượng, nhưng đó chính là kỹ năng cần luyện. Tốt hơn nữa là luyện theo cặp, thay nhau đóng vai người phỏng vấn. Ngoài ra, tự đặt giới hạn thời gian 40 phút cho mỗi bài để làm quen với sức ép.

*Cho thi đấu:* thi ảo là công cụ không thể thay thế. Lấy một đề cũ, đặt đồng hồ đúng thời lượng, không tra cứu, làm nghiêm túc. Sau đó ghi lại mốc thời gian mỗi lần đổi bài và đối chiếu với hai quyết định chiến thuật ở mục 2. Rồi upsolve theo quy trình ở A/02.

*Cho cả hai:* chuẩn bị sẵn hai thứ. Một *thư viện mẫu* đã kiểm chứng cho các cấu trúc hay dùng — DSU, cây phân đoạn, Dijkstra, luồng — vì thời gian gỡ lại từ đầu là thời gian không dùng để nghỉ. Và một *bảng kiểm trước khi nộp* gồm năm tới bảy dòng, rút từ nhật ký lỗi của chính bạn.

### 3.4 Gốc rễ: từ một cuộc thi ở Texas tới quy trình phỏng vấn hiện đại

Thi lập trình có tuổi đời khá dài. Các cuộc thi giữa các trường đại học bắt đầu từ đầu thập niên 1970 ở Mỹ và dần lớn thành giải vô địch lập trình sinh viên quốc tế; kỳ thi tin học quốc tế dành cho học sinh phổ thông bắt đầu năm 1989. Hai thể thức này khác nhau ở một điểm ảnh hưởng lớn tới chiến thuật: thi đồng đội theo kiểu ICPC chấm đúng/sai kèm phạt thời gian, còn IOI chấm từng phần theo nhóm test. Sự khác biệt đó tạo ra hai văn hoá luyện tập khác nhau — một bên nhấn mạnh tốc độ và độ chính xác lần nộp đầu, một bên nhấn mạnh khả năng lấy điểm bộ phận và tối ưu dần.

Phỏng vấn kỹ thuật có một lịch sử ngắn hơn và nhiều thay đổi hơn. Giai đoạn đầu của các công ty công nghệ lớn nổi tiếng với các câu đố mẹo — những câu hỏi kiểu “vì sao nắp cống hình tròn” — và cách làm đó dần bị bỏ khi có bằng chứng rằng chúng không dự báo được năng lực làm việc. Từ giữa thập niên 2000, các câu hỏi thuật toán có cấu trúc trở thành chuẩn, và bộ sách hướng dẫn phỏng vấn[8] đóng vai trò lớn trong việc chuẩn hoá kỳ vọng của cả hai phía.

Điều đáng nhớ về mặt phương pháp là vì sao quy trình bảy bước ở mục 1 lại được chăm cao. Nó không phải một nghi thức: mỗi bước tương ứng với một hành vi mà người phỏng vấn thực sự cần dự báo. Hỏi làm rõ trước khi làm dự báo việc bạn có xây sai thứ trong dự án thật hay không. Nói ra suy nghĩ dự báo việc cộng tác. Tự kiểm tra dự báo chất lượng mã bạn giao. Nhìn theo cách đó, buổi phỏng vấn là một mô phỏng thu nhỏ — và cách luyện tốt nhất là làm việc theo đúng quy trình đó cả khi không ai xem, cho tới khi nó thành phản xạ.

#### Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                                                    | Bài toán thúc ép lúc đó                                    | Điều thay đổi sau đó                                                                                            |
|--------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Đầu thập niên 1970 — thi lập trình sinh viên           | Cần một cách đánh giá kỹ năng lập trình mang tính thi đấu. | Thể thức đồng đội, chấm đúng/sai kèm phạt thời gian; định hình văn hoá coi trọng độ chính xác lần nộp đầu.      |
| 1989 — kỳ thi tin học quốc tế                          | Đánh giá học sinh phổ thông ở quy mô quốc tế.              | Chấm từng phần theo nhóm test; sinh ra chiến thuật lấy điểm bộ phận và tối ưu dần.                              |
| Thập niên 1990–2000 — phỏng vấn kiểu câu đố            | Cần lọc ứng viên nhanh ở quy mô lớn.                       | Câu đố mẹo phổ biến rồi bị bỏ vì không dự báo được năng lực làm việc thật.                                      |
| Giữa thập niên 2000 — phỏng vấn thuật toán có cấu trúc | Cần tiêu chí đánh giá lập lại được và công bằng hơn.       | Bài thuật toán 45 phút thành chuẩn; sách hướng dẫn chuẩn hoá kỳ vọng của cả hai phía[8].                        |
| Thập niên 2010 — nền tảng luyện tập trực tuyến         | Người học cần môi trường luyện có phản hồi tức thì.        | Kho bài phân loại theo độ khó và thi ảo; độ trễ phản hồi giảm mạnh, đúng yếu tố mà A/02 nói là quan trọng nhất. |

3.5 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực                       | Ý tưởng tương ứng                                 | Giống ở đâu, khác ở đâu                                                                                                                                                                            |
|--------------------------------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cờ vua                         | Quản lý đồng hồ, tránh áp lực thời gian           | Kỳ thủ mạnh không tính sâu hơn nhiều mà <i>phân bổ thời gian</i> tốt hơn — đúng phát hiện của Schoenfeld ở A/01. Cùng bài học: quyết định nhanh ở thể đơn giản để dành thời gian cho thể phức tạp. |
| Thể thao                       | Thi đấu tập, mô phỏng điều kiện thật              | Thi ảo chính là buổi tập có tính giờ. Nguyên tắc chung: kỹ năng chịu sức ép chỉ luyện được khi có sức ép, nên tập nhẹ nhàng không chuẩn bị cho ngày thi.                                           |
| Hàng không                     | Bảng kiểm, huấn luyện buồng lái mô phỏng          | Bảng kiểm trước khi nộp bài giống bảng kiểm trước cất cánh: không phải vì bạn không biết, mà để chống bỏ sót dưới áp lực. Buồng lái mô phỏng chính là thi ảo.                                      |
| Y khoa                         | Thi lâm sàng có cấu trúc, “nghĩ to” khi chẩn đoán | Sinh viên y được dạy diễn đạt lập luận chẩn đoán thành lời — vì cùng lý do với mục 1.2: người chấm cần thấy quá trình, và bệnh nhân cần hiểu lý do.                                                |
| Trình bày và bảo vệ nghiên cứu | Trả lời câu hỏi của hội đồng                      | Cùng ba lỗi giao tiếp: im lặng quá lâu, trả lời trước khi hiểu câu hỏi, và phòng thủ khi bị góp ý. Cách chữa cũng giống nhau.                                                                      |
| Tâm lý học thể thao            | Trạng thái mất bình tĩnh sau thất bại             | Hiện tượng nộp sai liên tiếp ở mục 2.1 là một dạng của nó. Biện pháp cũng tương tự: một thủ tục cố định để chạy sau mỗi thất bại, thay vì dựa vào ý chí.                                           |

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì lời giải cuối cùng nói rất ít về việc bạn sẽ làm việc thế nào. Bạn có thể đã gặp đúng bài đó, hoặc gặp một bài hoàn toàn xa lạ — hai trường hợp cho cùng kết quả nhưng nói hai điều khác nhau. Quá trình thì dự báo được: hỏi làm rõ trước khi làm dự báo bạn có xây sai thứ trong dự án thật hay không; nói ra suy nghĩ dự báo khả năng cộng tác; tự kiểm tra dự báo chất lượng mã bạn giao. Buổi phỏng vấn là một mô phỏng thu nhỏ của việc cùng làm việc.
- A2. Vì thứ tự đề không luôn theo độ khó, và bạn không thể phân bổ thời gian hợp lý cho thứ mình chưa biết. Mười phút đọc hết đề cho bạn một bức tranh để xếp thứ tự làm, và nó thường tiết kiệm được nửa giờ. Ngoài ra, việc đọc một bài rồi để đó có tác dụng phụ tốt: não bạn tiếp tục xử lý nó trong lúc bạn làm bài khác — hiện tượng ấp ủ ở A/01.
- A3. Quyết định đúng gần như luôn là chuyển sang bài khác và quay lại sau, kèm ghi một dòng về chỗ đang kẹt. Cách biết trước khi hết giờ là đặt mốc *ngay khi bắt đầu bài*: “nếu 25 phút nữa chưa có hướng rõ ràng thì chuyển”. Mốc phải đặt trước, vì khi đã bỏ 40 phút thì tâm lý tiếc công đã bỏ ra sẽ khiến bạn quyết định tệ. Đây chính là tăng tự giám sát mà A/01 mô tả, ở dạng cụ thể nhất.
- A4. Vì im lặng với bạn là suy nghĩ, còn với người phỏng vấn là không có thông tin — ba phút nghĩ tốt và ba phút bế tắc trông giống hệt nhau. Quan trọng hơn, im lặng làm họ mất cơ hội gợi ý đúng chỗ, mà gợi ý là một phần của quy trình chứ không phải sự nhân nhượng. Nếu cần yên tĩnh để nghĩ, hãy nói trước một câu: “cho tôi một phút cân nhắc về cấu trúc dữ liệu” — điều đó vừa giữ được sự yên tĩnh vừa giữ được liên lạc.
- A5. Vì bạn thường sửa triệu chứng chứ không sửa nguyên nhân: bạn đổi một dấu so sánh, thêm một trường hợp riêng, mà không biết vì sao sai. Cộng thêm áp lực làm sự chú ý thu hẹp và loại bỏ khả năng suy xét rộng. Cách chống là một thủ tục cố định: dừng 60 giây, trả lời hai câu — ca dữ liệu nào có thể sai, giả định nào của tôi có thể không đúng — và nếu không trả lời được thì viết stress test thay vì đoán tiếp (E/25).



**NÓI LẠI THẬT ĐƠN GIẢN**

Cả kỳ thi lẫn buổi phỏng vấn đều không đo mỗi lượng kiến thức. Chúng đo cách bạn xoay sở khi có đồng hồ chạy và có người nhìn.

Trong phỏng vấn, điều làm người nghe an tâm không phải bạn ra đáp án nhanh, mà là họ theo được bạn đang nghĩ gì. Hãy hình dung bạn đang chỉ đường cho ai đó: nếu bạn im lặng đi trước rồi bất ngờ dừng lại nói “tới rồi”, người ta không biết bạn có đi đúng đường không. Vì vậy hãy hỏi cho rõ trước khi đi, nói ra bạn định rẽ hướng nào, và khi tới nơi thì tự kiểm tra lại xem có đúng địa chỉ không. Ba việc đó quan trọng hơn việc đi nhanh.

Trong kỳ thi, sai lầm tốn kém nhất không phải làm sai một bài khó, mà là dành cả tiếng cho bài khó trong khi có một bài dễ hơn chưa đọc tới. Cách phòng rất đơn giản và ít người làm: đọc hết đề trước rồi mới chọn bài, và mỗi khi bắt đầu một bài thì tự hẹn giờ — nếu quá hạn mà chưa thấy hướng thì chuyển sang bài khác, ghi lại một dòng về chỗ đang kẹt.

Còn khi vừa nộp sai một lần, hãy cưỡng lại việc sửa ngay. Trong trạng thái sốt ruột, người ta thường sửa cái nhìn thấy chứ không sửa cái gây ra lỗi, nên lần nộp sau lại sai. Một phút dừng lại tự hỏi “dữ liệu nào có thể làm nó sai” rẻ hơn nhiều so với ba lần nộp hỏng.

**Một câu:** *Kiến thức quyết định bạn giải được bài nào; chiến thuật quyết định bạn kịp giải bao nhiêu bài trong số đó.*

**Chỗ không rút gọn được:** Áp lực thật không mô phỏng hoàn toàn được; thi ảo giúp nhiều nhưng không thay thế được kinh nghiệm dự thi thật.

**Active recall** (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                          | Trả lời                                                                                                                            |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| ●●● <b>Bảy bước phỏng vấn</b>                | Làm rõ → ví dụ → vết cạn → cải thiện → thống nhất trước khi viết → viết → tự kiểm.                                                 |
| ●●● <b>Điều thực sự được chấm</b>            | Không phải lời giải tối ưu, mà là: hỏi trước khi làm, nói ra suy nghĩ, phản ứng tốt với gợi ý, tự kiểm tra không cần nhắc.         |
| ●●● <b>Ba lỗi giao tiếp</b>                  | Im lặng suy nghĩ; nhảy vào viết code trước khi thống nhất hướng; phòng thủ khi được góp ý.                                         |
| ●●● <b>Ba quyết định chiến thuật thi đấu</b> | Đọc hết đề trước (5–10 phút); đặt mốc thời gian trước khi bắt đầu mỗi bài; biết khi nào bỏ và ghi lại chỗ kẹt.                     |
| ●●● <b>Sau khi nộp sai</b>                   | Dừng 60 giây, hỏi “ca nào có thể sai” và “giả định nào có thể không đúng”; không trả lời được thì viết stress test thay vì đoán.   |
| ●● <b>Thế thức quyết định chiến thuật</b>    | Có phạt thời gian: rà bảng kiểm thêm hai phút trước khi nộp. Chấm từng phần: nộp lời giải chậm lấy điểm bộ phận trước khi tối ưu.  |
| ●● <b>Cách luyện phỏng vấn</b>               | Giải bài với việc nói to toàn bộ suy nghĩ, kể cả khi ngồi một mình; luyện theo cặp; tự đặt giới hạn 40 phút mỗi bài.               |
| ●● <b>Cách luyện thi đấu</b>                 | Thi ảo đúng thời lượng, không tra cứu; ghi mốc thời gian mỗi lần đổi bài; upsolve sau đó theo quy trình ở A/02.                    |
| ●● <b>Hai thứ cần chuẩn bị trước</b>         | Thư viện mẫu đã stress test; bảng kiểm trước khi nộp rút từ nhật ký lỗi của chính bạn.                                             |
| ●● <b>Vì sao đặt mốc thời gian trước</b>     | Vì khi đã bỏ 40 phút, tâm lý tiếc công sẽ làm bạn quyết định tệ; mốc phải đặt lúc còn khách quan.                                  |
| ● <b>Hai thế thức, hai văn hoá</b>           | Chấm đúng/sai kèm phạt thời gian coi trọng độ chính xác lần đầu; chấm từng phần coi trọng khả năng lấy điểm bộ phận và tối ưu dần. |

●●● phải thuộc    ●● nên nắm    ● biết là có

**Câu hỏi ôn tập có đáp án** (che phần đáp án, tự trả lời trước)

1. Nếu tôi bí hoàn toàn trong phỏng vấn thì sao?



| Nói ra chính xác chỗ bạn kẹt, dạng “tôi biết cách làm nếu... nhưng chưa biết xử lý khi...” — đúng kỹ thuật ở A/01. Điều này có ba tác dụng: cho người phỏng vấn biết bạn đang ở đâu để gợi ý đúng chỗ; cho thấy bạn suy nghĩ có cấu trúc; và rất thường xuyên, chính lúc phát biểu ra thì bạn nhìn thấy hướng đi. Điều nên tránh là im lặng, và điều tuyệt đối nên tránh là giả vờ đang tiến triển.

## 2. Có nên nói thẳng rằng tôi đã từng gặp bài này không?

| Nên, và hầu hết người phỏng vấn đánh giá cao sự thẳng thắn đó. Họ thường sẽ đổi bài, hoặc yêu cầu bạn giải thích sâu hơn — cả hai đều tốt hơn cho bạn so với việc giả vờ nghĩ ra rồi bị hỏi một biến thể mà bạn không hiểu bản chất. Ngoài ra, việc giả vờ rất dễ lộ: người có kinh nghiệm nhận ra ngay khi ứng viên đi thẳng tới lời giải tối ưu mà không qua bước nào.

## 3. Trong contest, tôi nên đọc đề theo thứ tự nào?

| Đọc lướt toàn bộ theo thứ tự đề, ghi cho mỗi bài một ước lượng độ khó và một dòng về ý tưởng nếu đã thấy. Sau đó làm theo thứ tự khó tăng dần *theo ước lượng của bạn*, không theo thứ tự đề. Nếu có bảng điểm trực tuyến, hãy dùng số người đã giải làm nguồn thông tin — nó chính xác hơn cảm giác cá nhân, đặc biệt với những bài mà đề dài làm bạn thấy đáng sợ hơn thực tế.

## 4. Tôi nên chuẩn bị thư viện mẫu tới mức nào?

| Đủ để không phải gõ lại những thứ dài và dễ sai: DSU, cây phân đoạn có lazy, Dijkstra, luồng cực đại, hình học cơ bản, số học modulo. Nguyên tắc quan trọng nhất: mỗi đoạn phải *do bạn viết* và đã stress test — chép về mà không hiểu thì khi cần sửa cho biến thể lạ bạn sẽ kẹt (A/02). Đừng nhồi quá nhiều: một thư viện mà bạn không nhớ có gì trong đó thì vô dụng.

## 5. Phỏng vấn thiết kế hệ thống có liên quan gì tới quyển sách này không?

| Có, ở phần tư duy đánh đổi chứ không ở phần thuật toán cụ thể. Các câu hỏi ở đó xoay quanh đúng những chủ đề của E/26: chọn cấu trúc theo mẫu truy cập, ước lượng quy mô bằng phép tính phong bì, độ trễ đuôi, đánh đổi giữa nhất quán và khả dụng. Kiến thức trực tiếp dùng lại: băm nhất quán, B-cây và chỉ mục, hàng đợi, và các cấu trúc xấp xỉ cho luồng dữ liệu.

## 6. Tôi luyện nhiều mà thi vẫn kém. Nguyên nhân hay gặp là gì?

| Ba nguyên nhân, và cả ba đều thuộc về chiến thuật chứ không kiến thức. Một, chưa bao giờ luyện có tính giờ — nên kỹ năng dưới sức ép chưa được tập. Hai, không đọc hết đề nên chọn sai bài. Ba, không có bảng kiểm nên mất điểm vào lỗi cài đặt lặp đi lặp lại. Cách chẩn đoán: sau vài kỳ thi, phân loại các bài bạn *không* giải được thành “không nghĩ ra”, “nghĩ ra nhưng không kịp”, và “làm sai”. Tỉ lệ ba nhóm cho biết bạn cần luyện gì.

## 7. Có nên luyện gõ code nhanh không?

| Ở mức vừa phải thì có, nhưng nó không phải nút thắt của phần lớn người học. Điều đáng luyện hơn là gõ *ít lỗi*: viết bất biến trước, giữ quy ước nhất quán, và có thư viện mẫu để không phải gõ lại thứ dài. Nếu bạn thấy mình thường xuyên hết giờ khi đang gõ chứ không khi đang nghĩ, thì hãy xem lại xem vấn đề có thật sự là tốc độ gõ hay là bạn viết lại từ đầu những thứ lẽ ra đã có sẵn.

## | Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                               | Nguồn                        | Điều cần nhận ra                                                                     |
|-------------------------------------------------------------------|------------------------------|--------------------------------------------------------------------------------------|
| ● Giải một bài quen thuộc và nói to toàn bộ suy nghĩ, có ghi âm   | Bài tập giao tiếp            | Nghe lại và đếm số lần im lặng quá 30 giây; đó là con số cần giảm.                   |
| ● Chạy quy trình bảy bước cho một bài dễ, tính giờ 40 phút        | Bài tập quy trình            | Mục tiêu là đủ bảy bước, không phải ra lời giải tối ưu.                              |
| ●● Thi ảo một đề contest cũ, đúng thời lượng                      | Codeforces, AtCoder, CSES[9] | Ghi mốc thời gian mỗi lần đổi bài; đối chiếu với hai quyết định chiến thuật ở mục 2. |
| ●● Phân loại các bài bạn không giải được trong ba kỳ thi gần nhất | Bài tập tự chẩn đoán         | Ba nhóm: không nghĩ ra, không kịp, làm sai. Tỉ lệ cho biết nên luyện gì.             |

| Bài                                                                    | Nguồn              | Điều cần nhận ra (tiếp)                                                              |
|------------------------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------|
| ●● Luyện phỏng vấn theo cặp, thay nhau đóng vai người phỏng vấn        | Bài tập giao tiếp  | Đóng vai người phỏng vấn dạy nhiều không kém đóng vai ứng viên.                      |
| ●● Lập bảng kiểm trước khi nộp gồm 5–7 dòng từ nhật ký lỗi của bạn     | Bài tập tự tổ chức | Dùng nó trong ba kỳ thi tiếp theo và đếm số lần nó cứu bạn.                          |
| ●●● Xây thư viện mẫu cho sáu cấu trúc hay dùng, mỗi cái có stress test | Bài tập cài đặt    | DSU, cây phân đoạn lazy, Dijkstra, luồng, hình học cơ bản, số học modulo.            |
| ●●● Thi ảo với ràng buộc chỉ được nộp mỗi bài một lần                  | Bài tập kỷ luật    | Luyện thói quen rà bảng kiểm; đo xem tỉ lệ đúng lần đầu của bạn là bao nhiêu.        |
| ●●● Upsolve toàn bộ một đề contest cho tới khi giải hết                | Bài tập upsolving  | Đây là bài tập tạo tiến bộ lớn nhất trong danh sách, và cũng là bài tập ít vui nhất. |

## Kết chương và đọc tiếp

Chương này về hai bối cảnh có sức ép, và luận điểm chính là chúng đo những thứ luyện được mà người học thường không luyện. Ba điều đáng mang theo: quy trình bảy bước cho phỏng vấn, trong đó bước làm rõ đề và bước tự kiểm là hai bước tạo khác biệt lớn nhất; ba quyết định chiến thuật cho thi đấu, đặc biệt việc đặt mốc thời gian *trước* khi bắt đầu một bài; và thủ tục 60 giây sau mỗi lần nộp sai.

Chương cuối cùng gom toàn bộ quyển sách thành thứ bạn mang theo được: một lộ trình luyện tập theo tuần, một cách theo dõi tiến bộ, và một ngân hàng bài tập phân loại theo 27 chương trước — để sau khi gấp sách, bạn có việc cụ thể để làm vào sáng mai ([E/28](#)).

# Nguồn

---

- [1] A. Zeller and R. Hildebrandt, "Simplifying and Isolating Failure-Inducing Input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [2] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [3] D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [4] J. L. Bentley and M. D. McIlroy, "Engineering a Sort Function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [5] D. R. Musser, "Introspective Sorting and Selection Algorithms," *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [6] J. Bentley, *Writing Efficient Programs*. Prentice-Hall, 1982.
- [7] F. McSherry, M. Isard, and D. G. Murray, "Scalability! But at What COST?" In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [8] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [9] A. Laaksonen. "CSES Problem Set." (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [10] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [11] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [12] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [13] J. Erickson, *Algorithms*. Independently published, 2019.

## Lộ trình luyện tập và ngân hàng bài tập (Training Plan and Problem Bank)

**PHỤ THUỘC** A/02 cho nguyên lý luyện tập có chủ đích và vòng upsolving; E/27 cho việc luyện dưới sức ép. Đây là chương bạn sẽ mở lại nhiều lần nhất.

### TÓM TẮT

Chương cuối biến 27 chương trước thành thứ mang theo được: một lịch luyện cụ thể, một cách theo dõi tiến bộ, và một ngân hàng bài tập phân loại theo chương. Nội dung gồm ba phần. Phần một là cấu trúc một buổi luyện và một tuần luyện, cùng bốn công cụ theo dõi — nhật ký lỗi, danh sách mẫu, lịch ôn ngắt quãng, và ba chỉ số tiến bộ. Phần hai là ba lộ trình theo mục tiêu: 12 tuần cho phòng vắn, 6 tháng cho thi đấu, và một nhịp duy trì cho người đi làm. Phần ba là ngân hàng bài tập được nhóm theo chủ đề, đủ dùng cho vài tháng. Nếu chỉ nhớ một điều: tiến bộ đến từ số bài bạn *giải xong sau khi đã bí*, nên hãy thiết kế lịch luyện quanh việc upsolving chứ không quanh số bài đã đọc lời giải.

### KIẾN THỨC CẦN CÓ TRƯỚC

- **buổi luyện** — một phiên có mục tiêu hẹp, thời lượng cố định và một cách kiểm tra kết quả — khác với “ngồi giải bài tới khi chán”.
- **ôn ngắt quãng** — ôn lại một nội dung ở các khoảng cách tăng dần (1 ngày, 3 ngày, 1 tuần, 1 tháng), vì trí nhớ được củng cố mạnh nhất ngay trước lúc sắp quên.
- **danh sách mẫu** — sổ ghi các ý tưởng cốt lõi ở dạng tổng quát, mỗi mẫu một câu không nhắc tới đề bài cụ thể (A/01).
- **nhật ký lỗi** — với mỗi lần sai: hiểu sai điều gì, dấu hiệu nào lẽ ra đã gợi ý đúng, lần sau kiểm tra gì.
- **vùng luyện** — mức khó mà bạn giải được khoảng một nửa số bài; dễ hơn thì không học được gì, khó hơn thì không động lại (A/02).
- **chỉ số tiến bộ** — đại lượng đo được để biết mình có tiến không — ví dụ tỉ lệ giải được ở một mức khó cố định, chứ không phải cảm giác.

### CÂU HỎI MỞ ĐẦU

- Q1. Vì sao “giải càng nhiều bài càng tốt” lại là lời khuyên nguy hiểm?
- Q2. Nếu mỗi ngày chỉ có một giờ, phân bổ thế nào cho hiệu quả nhất?
- Q3. Làm sao biết mình đang tiến bộ, khi cảm giác chủ quan gần như luôn sai?
- Q4. Nên học kiến thức mới hay giải bài nhiều hơn — và tỉ lệ nào là hợp lý?
- Q5. Vì sao lộ trình nào cũng đổ vỡ sau vài tuần, và cách thiết kế nào chống lại điều đó?

Sau khi đọc xong 27 chương, câu hỏi thực tế duy nhất còn lại là: sáng mai làm gì. Chương này trả lời câu đó.

Nguyên tắc nền đã được nêu ở A/02 và đáng nhắc lại vì mọi thứ trong chương này đều suy ra từ nó: luyện tập chỉ tạo tiến bộ khi có mục tiêu hẹp, mức khó hơi vượt khả năng, phản hồi nhanh,

và có sửa sai. Bốn điều kiện đó biến thành bốn quyết định thiết kế: buổi luyện phải có mục tiêu viết ra được; bài phải chọn ở vùng bạn giải được khoảng một nửa; phải có chấm bài hoặc stress test; và phải có nhật ký lỗi cùng bước upsolving. Lịch luyện nào thiếu một trong bốn thì tiêu tốn thời gian mà không đổi được gì.

#### 4.1 Cấu trúc một buổi và một tuần

Một buổi 60 phút có cấu trúc dùng được như sau: 10 phút ôn — đọc lại nhật ký lỗi và một bảng active recall của chương đang học; 40 phút giải một bài ở vùng luyện, có bấm giờ; 10 phút cho bước bốn — viết một câu về ý tưởng cốt lõi vào danh sách mẫu, và ghi nhật ký nếu sai. Tỷ lệ có thể xê dịch, nhưng đừng cắt 10 phút cuối: đó là phần biến một lần giải bài thành kiến thức dùng lại được.

Một tuần nên có ba loại buổi khác nhau, và sự xen kẽ này quan trọng hơn tổng số giờ. *Buổi kiến thức*: đọc một mục của quyển sách này hoặc một chủ đề mới, rồi cài thử ngay và cho vào thư viện. *Buổi bài tập*: ba tới năm bài cùng một chủ đề, để nhận diện thành phần xạ. *Buổi thi*: một kỳ thi ảo tính giờ, xen kẽ nhiều chủ đề, kèm upsolving sau đó. Tỷ lệ hợp lý cho người mới là 2:3:1 mỗi tuần; càng về sau càng nghiêng về buổi thi.

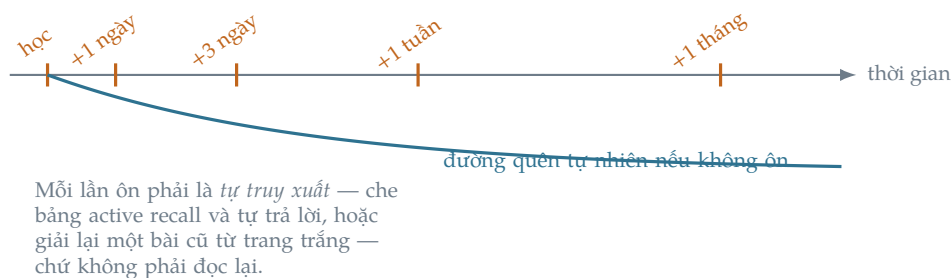
Về câu hỏi “học kiến thức mới hay giải bài”, tỷ lệ thường dùng là khoảng một phần ba thời gian cho kiến thức mới và hai phần ba cho bài tập — và tỷ lệ đó nên dịch dần về phía bài tập. Lý do: kiến thức không có nhu cầu đi kèm sẽ bay hết trong hai tuần, còn bài tập thì vừa củng cố cái cũ vừa lộ ra cái còn thiếu.

#### 4.2 Bốn công cụ theo dõi

*Nhật ký lỗi*. Ba dòng cho mỗi lần sai. Sau một tháng, phân loại chúng thành ba nhóm: hiểu sai đề, sai ý tưởng, sai cài đặt. Tỷ lệ ba nhóm nói cho bạn biết phải luyện gì — và nó thường gây bất ngờ, vì phần lớn người học nghĩ mình yếu ý tưởng trong khi thực tế mất điểm nhiều nhất ở hai nhóm kia.

*Danh sách mẫu*. Mỗi bài giải xong, một câu tổng quát không nhắc đề bài. “Khi tìm cực trị trên mọi đoạn, cố định điểm cuối và duy trì đáp án tốt nhất kết thúc tại đó.” Sau vài tháng, danh sách này là tài sản có giá trị nhất bạn tích lũy được, và nó chính là các *khô tri thức* mà nghiên cứu về chuyên gia cờ vua mô tả (A/02).

*Lịch ôn ngắt quãng*. Với mỗi chủ đề đã học, ôn lại sau 1 ngày, 3 ngày, 1 tuần, 1 tháng. “Ôn” ở đây nghĩa là *tự truy xuất*: che bảng active recall và tự trả lời, hoặc giải một bài cũ từ trang trắng — không phải đọc lại.



**Hình 4.1.** Lịch ôn ngắt quãng. Các mốc đặt ở khoảng cách tăng dần vì trí nhớ được củng cố mạnh nhất khi việc truy xuất khó nhưng vẫn thành công: ôn quá sớm thì tốn công, ôn quá muộn thì phải học lại từ đầu.

Ba chỉ số tiến bộ. Cảm giác chủ quan gần như luôn sai, nên hãy đo. Chỉ số một: tỉ lệ giải được ở một mức khó cố định — nếu mức khó cũng tăng thì bạn không so được. Chỉ số hai: thời gian trung bình để giải một bài ở mức bạn đã quen. Chỉ số ba, và là chỉ số tốt nhất: số bài bạn *upsolve* tới nơi tới chốn mỗi tuần.

#### CẦN NHỚ

Ba việc chiếm chưa tới mười phút mỗi buổi nhưng tạo ra phần lớn tiến bộ: viết một câu mẫu sau mỗi bài giải xong; ghi ba dòng nhật ký sau mỗi lần sai; và *upsolve* ít nhất một bài mỗi tuần từ trang trắng. Nếu phải cắt lịch luyện vì bận, hãy cắt phần đọc kiến thức mới chứ đừng cắt ba việc này.

### 4.3 Ba lộ trình theo mục tiêu

#### Lộ trình 12 tuần — chuẩn bị phỏng vấn

Giả định 8–10 giờ mỗi tuần. Mục tiêu: giải được một bài mức trung bình trong 35 phút với quy trình bảy bước ở E/27.

| Tuần | Nội dung                                       | Tiêu chí kiểm tra khách quan                             |
|------|------------------------------------------------|----------------------------------------------------------|
| 1    | Chương A/01, A/03; mảng, hai con trỏ (B/06)    | Thuộc bảng giới hạn → độ phức tạp; giải 10 bài dễ        |
| 2    | Băm, tập hợp (B/08); ngăn xếp, hàng đợi (B/07) | Nhận ra dạng cửa sổ trượt trong 2 phút; 12 bài           |
| 3    | Sắp xếp, tìm kiếm nhị phân (C/14, C/19)        | Viết nhị phân đúng từ trang trắng, có stress test        |
| 4    | Cây và đệ quy (B/09); duyệt cây                | 10 bài cây; viết được duyệt trước/giữa/sau không cần tra |
| 5    | Đồ thị: duyệt rộng, duyệt sâu (C/17)           | Nhận ra bài đồ thị trá hình; 10 bài                      |
| 6    | Đồ thị: đường đi ngắn nhất, tô-pô              | Chọn đúng thuật toán theo giả thiết trọng số             |
| 7    | Quy hoạch động một chiều và ba lô (C/16)       | Viết được năm bước ra giấy trước khi gõ, mọi bài         |
| 8    | Quy hoạch động hai chuỗi và trên cây           | 10 bài; tự phát hiện được trạng thái thiếu               |
| 9    | Tham lam (C/15); heap                          | Chứng minh hoặc bác bỏ được tiêu chí bằng stress test    |
| 10   | Ôn tập trộn chủ đề; E/25                       | 3 buổi thi ảo trộn chủ đề, mỗi buổi 2 bài trong 70 phút  |
| 11   | Phỏng vấn thử theo cặp (E/27)                  | 4 buổi phỏng vấn thử; ghi âm và nghe lại                 |
| 12   | Ôn nhật ký lỗi và danh sách mẫu                | Giải lại 10 bài cũ từ trang trắng, đạt trên 80%          |

Lưu ý về cách dùng bảng này: cột thứ ba mới là phần quan trọng. Một tuần “xong” không phải khi bạn đọc hết nội dung mà khi bạn đạt tiêu chí kiểm tra — nếu chưa đạt thì kéo dài thêm vài ngày thay vì chạy tiếp.

#### Lộ trình 6 tháng — thi đấu

Giả định 10–15 giờ mỗi tuần. Cấu trúc theo tháng: tháng 1–2 phủ phần B (cấu trúc dữ liệu) và các chương C/13–C/16; tháng 3–4 phủ C/17–C/20 và D/21; tháng 5 phủ B/10, B/12, D/22; tháng 6 dành hoàn toàn cho thi ảo và *upsolving*.

Mỗi tuần trong sáu tháng đó nên có: một kỳ thi ảo tính giờ; *upsolving* toàn bộ đề đó cho tới khi giải hết hoặc hiểu hết; và hai buổi chủ đề. Đây là nhịp mà phần lớn người tiến bộ nhanh đều theo,

và điểm khác biệt của họ so với người luyện nhiều mà không tiến thường nằm ở chữ “toàn bộ” trong bước upsolving.

### Nhịp duy trì — người đi làm

Nếu mục tiêu là không quên và tiến chậm nhưng đều: một bài mỗi tuần ở vùng luyện, cộng một kỳ thi ảo mỗi tháng, cộng đọc lại danh sách mẫu mỗi tháng. Tổng khoảng hai giờ mỗi tuần. Điều quan trọng hơn khối lượng là tính liên tục — và câu trả lời cho câu hỏi mở đầu thứ năm nằm ở đây: lộ trình đổ vỡ vì đặt mục tiêu theo ngày bận nhất chứ không theo ngày bình thường. Hãy đặt mức tối thiểu mà bạn chắc chắn làm được kể cả ngày tệ nhất, rồi làm thêm khi rảnh.

## 4.4 Ngân hàng bài tập

Phần này gom các bài theo chủ đề. Nguồn được ghi bằng tên bài để bạn tra; các tập bài được nhắc tới gồm CSES[9], chuỗi bài quy hoạch động của AtCoder, và các bài kinh điển trên LeetCode. Cách dùng: chọn một chủ đề, làm theo thứ tự mức độ, và upsolve mọi bài không giải được trong 60 phút.

### Ngân hàng — Nền tảng và mảng (B/06) (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                                     | Nguồn                | Điều cần nhận ra                                                     |
|-------------------------------------------------------------------------|----------------------|----------------------------------------------------------------------|
| ● Increasing Array, Missing Number, Repetitions                         | CSES Introductory[9] | Khởi động; luyện đọc đề và xử lý ca biên.                            |
| ● Distinct Numbers, Apartments, Ferris Wheel                            | CSES Sorting[9]      | Sắp xếp như nước đi mở đầu; hai con trỏ ở dạng đơn giản nhất.        |
| ●● Sum of Two Values, Sum of Three Values, Sum of Four Values           | CSES Sorting[9]      | Ba mức của cùng một ý: cố định bớt biến rồi hai con trỏ hoặc băm.    |
| ●● Maximum Subarray Sum, Subarray Sums I và II                          | CSES Sorting[9]      | Tổng tiền tố; mẫu “điều kiện đoạn thành điều kiện hai điểm nút”.     |
| ●● Playlist, Traffic Lights, Room Allocation                            | CSES Sorting[9]      | Cửa sổ trượt và tập hợp có thứ tự; kiểm tính đơn điệu trước khi cài. |
| ●●● Subarray Divisibility, Subarray Distinct Values, Array Division     | CSES Sorting[9]      | Ba hướng khác nhau: modulo, cửa sổ, nhị phân trên đáp án.            |
| ●●● Trapping Rain Water, Longest Substring Without Repeating Characters | LeetCode             | Hai bài phỏng vấn kinh điển; giải mỗi bài bằng hai cách.             |

### Ngân hàng — Ngăn xếp, hàng đợi, heap (B/07) (mức: ● khởi động ●● chuẩn ●●● khó)

| Bài                                                   | Nguồn             | Điều cần nhận ra                                              |
|-------------------------------------------------------|-------------------|---------------------------------------------------------------|
| ● Valid Parentheses, Min Stack                        | LeetCode          | Ngăn xếp cơ bản; chú ý ca dư ngoặc mở.                        |
| ●● Nearest Smaller Values                             | CSES Sorting[9]   | Ngăn xếp đơn điệu chuẩn; viết bất biến trước.                 |
| ●● Sliding Window Maximum, Sliding Window Median      | LeetCode; CSES[9] | Deque đơn điệu và hai heap; so hai công cụ.                   |
| ●● Josephus Problem I và II, Towers                   | CSES Sorting[9]   | Cấu trúc phù hợp quyết định độ dài code.                      |
| ●●● Largest Rectangle in Histogram, Maximal Rectangle | LeetCode          | Ngăn xếp đơn điệu ở mức khó; bài thứ hai quy về bài thứ nhất. |



| Bài                                                      | Nguồn           | Điều cần nhận ra (tiếp)                    |
|----------------------------------------------------------|-----------------|--------------------------------------------|
| ••• Tasks and Deadlines, Reading Books, Factory Machines | CSES Sorting[9] | Tham lam kèm heap và nhị phân trên đáp án. |

### Ngân hàng – Truy vấn đoạn và DSU (B/10, B/11) (mức: • khởi động •• chuẩn ••• khó)

| Bài                                                                            | Nguồn                 | Điều cần nhận ra                                                 |
|--------------------------------------------------------------------------------|-----------------------|------------------------------------------------------------------|
| • Static Range Sum Queries, Static Range Minimum Queries                       | CSES Range Queries[9] | Tổng tiền tố và sparse table; hiểu vì sao không hoán đổi được.   |
| •• Dynamic Range Sum Queries, Dynamic Range Minimum Queries, Range Xor Queries | CSES Range Queries[9] | Fenwick và cây phân đoạn; chú ý phần tử trung hoà.               |
| •• Range Update Queries, Hotel Queries, List Removals                          | CSES Range Queries[9] | Mảng hiệu, đi xuống trên cây, và cây chỉ số cho vị trí thứ $k$ . |
| •• Road Construction, Road Reparation, Building Roads                          | CSES Graph[9]         | DSU và cây khung nhỏ nhất; đếm số nhóm sau mỗi cạnh.             |
| ••• Range Updates and Sums, Polynomial Queries, Subarray Sum Queries           | CSES Range Queries[9] | Lazy propagation; bài cuối cần thiết kế phép gộp bốn giá trị.    |
| ••• Distinct Values Queries, Salary Queries, Forest Queries II                 | CSES Range Queries[9] | Nén toạ độ, xử lý ngoại tuyến, và cấu trúc hai chiều.            |
| ••• New Roads Queries, Dynamic Connectivity                                    | CSES Advanced[9]      | DSU với thời gian; kết hợp B/10 và B/11.                         |

### Ngân hàng – Quy hoạch động (C/16) (mức: • khởi động •• chuẩn ••• khó)

| Bài                                                                   | Nguồn                  | Điều cần nhận ra                                                            |
|-----------------------------------------------------------------------|------------------------|-----------------------------------------------------------------------------|
| • Frog 1, Frog 2, Vacation                                            | AtCoder Educational DP | Ba bài nhập môn; viết đủ năm bước ra giấy.                                  |
| • Dice Combinations, Minimizing Coins, Coin Combinations I và II      | CSES DP[9]             | Hai bài cuối khác nhau đúng ở thứ tự vòng lặp — giải thích được vì sao.     |
| •• Knapsack 1, Knapsack 2, LCS                                        | AtCoder Educational DP | Ba lô hai chiều và họ hai chuỗi; chú ý mẹo đảo vai trò ở Knapsack 2.        |
| •• Book Shop, Array Description, Edit Distance, Removal Game          | CSES DP[9]             | Bốn họ trạng thái khác nhau trong cùng một buổi.                            |
| •• Grid 1, Coins, Sushi, Deque                                        | AtCoder Educational DP | Đếm, xác suất, kỳ vọng, và trò chơi — bốn kiểu mục tiêu.                    |
| ••• Increasing Subsequence, Projects, Elevator Rides, Counting Towers | CSES DP[9]             | Tăng tốc bằng cấu trúc dữ liệu và quy hoạch động mặt nạ bit.                |
| ••• Matching, Independent Set, Subtree, Grouping                      | AtCoder Educational DP | Mặt nạ bit và quy hoạch động trên cây, mức khó.                             |
| ••• Digit Sum, Permutation, Intervals, Tower                          | AtCoder Educational DP | Quy hoạch động trên chữ số và các bài cần sắp xếp trước khi quy hoạch động. |

### Ngân hàng – Đồ thị (C/17, C/18) (mức: • khởi động •• chuẩn ••• khó)

| Bài                                                             | Nguồn            | Điều cần nhận ra                                                  |
|-----------------------------------------------------------------|------------------|-------------------------------------------------------------------|
| • Counting Rooms, Labyrinth, Building Teams                     | CSES Graph[9]    | Duyệt rộng và duyệt sâu; truy vết đường đi.                       |
| •• Message Route, Round Trip, Monsters                          | CSES Graph[9]    | Duyệt rộng nhiều nguồn và phát hiện chu trình.                    |
| •• Shortest Routes I và II, Flight Discount, High Score         | CSES Graph[9]    | Dijkstra, Floyd–Warshall, đồ thị nhiều tầng, và chu trình âm.     |
| •• Course Schedule, Longest Flight Route, Game Routes           | CSES Graph[9]    | Quy hoạch động theo thứ tự tô-pô.                                 |
| ••• Planets and Kingdoms, Giant Pizza, Flight Routes Check      | CSES Graph[9]    | Thành phần liên thông mạnh và ứng dụng cho bài logic hai giá trị. |
| ••• Company Queries I và II, Distance Queries, Counting Paths   | CSES Tree[9]     | Nhảy nhị phân, tổ tiên chung, và kỹ thuật hiệu trên cây.          |
| ••• Download Speed, Police Chase, School Dance, Distinct Routes | CSES Graph[9]    | Luồng cực đại, lát cắt nhỏ nhất, ghép đôi, và truy vết đường.     |
| ••• Necessary Roads, Necessary Cities                           | CSES Advanced[9] | Cầu và khớp; chú ý cạnh song song.                                |

#### Ngân hàng – Chuỗi, số học, hình học (B/12, D/21, D/22) (mức: • khởi động •• chuẩn ••• khó)

| Bài                                                                    | Nguồn            | Điều cần nhận ra                                             |
|------------------------------------------------------------------------|------------------|--------------------------------------------------------------|
| • Word Combinations, String Matching                                   | CSES String[9]   | Trie và so khớp; làm bằng cả băm lẫn hàm Z.                  |
| •• Finding Borders, Finding Periods, Longest Palindrome                | CSES String[9]   | Hàm tiền tố và Manacher.                                     |
| ••• Distinct Substrings, Repeating Substring, Substring Order I        | CSES String[9]   | Mảng hậu tố và LCP; bài cuối là lý do rõ nhất để học chúng.  |
| • Exponentiation, Counting Divisors                                    | CSES Math[9]     | Luỹ thừa nhanh và sàng lưu ước nhỏ nhất.                     |
| •• Binomial Coefficients, Distributing Apples, Bracket Sequences I     | CSES Math[9]     | Tổ hợp modulo và mô hình chia kẹo.                           |
| ••• Fibonacci Numbers, Graph Paths I, Exponentiation II                | CSES Math[9]     | Luỹ thừa ma trận và định lý Euler cho số mũ lớn.             |
| • Point Location Test, Polygon Area                                    | CSES Geometry[9] | Vị từ định hướng và công thức dây giày, hoàn toàn số nguyên. |
| •• Line Segment Intersection, Point in Polygon, Polygon Lattice Points | CSES Geometry[9] | Xử lý suy biến; kiểm bằng tọa độ nhỏ.                        |
| ••• Convex Hull, Minimum Euclidean Distance                            | CSES Geometry[9] | Quét đơn điệu và chia để trị.                                |

#### Ngân hàng – Tổng hợp và kỹ năng nghề (mức: • khởi động •• chuẩn ••• khó)

| Bài                                                          | Nguồn                          | Điều cần nhận ra                                              |
|--------------------------------------------------------------|--------------------------------|---------------------------------------------------------------|
| •• Apple Division, Meet in the Middle                        | CSES Introductory, Advanced[9] | Duyệt tập con và gặp nhau ở giữa — nhận ra qua giới hạn $n$ . |
| •• Viết khung stress test và dùng nó tìm một phần ví dụ thật | E/25                           | Công cụ dùng lại cả đời; đây là bài tập có lợi ích lâu nhất.  |

| Bài                                                                         | Nguồn                 | Điều cần nhận ra (tiếp)                                            |
|-----------------------------------------------------------------------------|-----------------------|--------------------------------------------------------------------|
| •• Xây thư viện mẫu sáu cấu trúc, mỗi cái có stress test                    | E/27                  | DSU, cây phân đoạn lazy, Dijkstra, luồng, hình học, số học modulo. |
| ••• Thi ảo một đề cũ, upsolve toàn bộ                                       | Codeforces / AtCoder  | Lập lại hằng tuần; đây là nguồn tiến bộ chính.                     |
| ••• Chọn 10 bài đã giải và viết một câu mẫu cho mỗi bài                     | Danh sách mẫu của bạn | Kiểm tra: câu đó có dùng lại được cho bài khác không.              |
| ••• Đọc mã nguồn một thư viện bạn dùng và tìm một thuật toán trong sách này | E/26                  | So bản kỹ thuật hoá với bản trong sách; ghi lại khác biệt.         |

#### 4.5 Gốc rễ: vì sao lộ trình lại có hình dạng như vậy

Các lựa chọn thiết kế trong chương này không tùy tiện; mỗi cái đến từ một kết quả đã nhắc ở A/02.

Việc *xen kẽ chủ đề* thay vì học tuần tự từng khối đến từ một phát hiện lặp lại trong nghiên cứu về học tập: luyện xen kẽ nhiều loại bài cho kết quả kém hơn khi đo ngay sau buổi tập, nhưng tốt hơn rõ rệt khi đo sau vài tuần. Lý do là luyện tuần tự cho phép bạn dùng cùng một phương pháp cho cả loạt bài mà không phải *chọn* phương pháp — và chọn phương pháp mới là kỹ năng cần cho bài thật. Đây cũng là lý do mục 1 khuyên có buổi thi trộn chủ đề mỗi tuần.

Việc *ôn ngắt quãng* đến từ một quan sát còn cổ hơn: trí nhớ được củng cố mạnh nhất khi việc truy xuất là khó nhưng vẫn thành công — tức là ngay trước lúc sắp quên. Ôn quá sớm thì tốn công mà không thêm gì; ôn quá muộn thì phải học lại. Các khoảng cách tăng dần là cách xấp xỉ thực dụng cho điều đó.

Việc đặt *upsolving* làm trung tâm đến từ cơ chế đã giải thích ở A/02: trong lúc vật lộn, bạn tạo ra những lỗ hổng rất cụ thể trong hiểu biết, và thông tin mới chỉ dính khi có lỗ hổng để lấp. Đó cũng là lý do chỉ số tiến bộ tốt nhất không phải số bài đã đọc lời giải mà là số bài đã upsolve tới nơi.

Cuối cùng, việc đặt *mức tối thiểu rất thấp* cho nhịp duy trì đến từ một quan sát thực dụng chứ không phải nghiên cứu: lộ trình hổng không phải vì người ta lười mà vì họ thiết kế cho ngày lý tưởng. Một lịch đòi hai giờ mỗi ngày sẽ đứt ở tuần thứ ba, và cái đứt đó tốn hơn nhiều so với việc chỉ làm ba mươi phút. Tính liên tục có giá trị kép: nó giữ trí nhớ, và nó giữ thói quen — thứ khó xây lại hơn kiến thức.

#### Gốc rễ: ý tưởng này ra đời từ bài toán nào

| Mốc                         | Bài toán thúc ép lúc đó                             | Điều thay đổi sau đó                                                                                                        |
|-----------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Nghiên cứu về luyện xen kẽ  | Học tuần tự từng khối cho cảm giác tiến nhanh hơn.  | Xen kẽ kém hơn khi đo ngay, tốt hơn khi đo sau vài tuần — vì nó buộc bạn <i>chọn</i> phương pháp, kỹ năng cần cho bài thật. |
| Nghiên cứu về ôn ngắt quãng | Ôn lại ngay thì tốn công, ôn muộn thì phải học lại. | Củng cố mạnh nhất khi truy xuất khó nhưng vẫn thành công; khoảng cách tăng dần là cách xấp xỉ thực dụng.                    |
| 1993 — Ericsson và cộng sự  | Vì sao cùng số giờ mà người tiến người không.       | Bốn điều kiện của luyện có chủ đích; toàn bộ thiết kế của chương này suy ra từ đó (A/02).                                   |
| Văn hoá thi đấu hiện đại    | Người luyện nhiều mà không tiến.                    | Upsolving tới nơi tới chốn thành chuẩn; số bài upsolve là chỉ số tốt hơn số bài đã xem lời giải.                            |

| Mốc                                      | Bài toán thúc ép lúc đó      | Điều thay đổi sau đó (tiếp)                                                                         |
|------------------------------------------|------------------------------|-----------------------------------------------------------------------------------------------------|
| Kinh nghiệm thực tế về duy trì thói quen | Lộ trình đồ vỡ sau vài tuần. | Đặt mức tối thiểu theo ngày tệ nhất chứ không theo ngày lý tưởng; tính liên tục quý hơn khối lượng. |

## 4.6 Liên hệ ngang

### Liên hệ ngang: ý này còn sống ở đâu nữa

| Lĩnh vực            | Ý tưởng tương ứng                                   | Giống ở đâu, khác ở đâu                                                                                                                                                             |
|---------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Thể thao            | Chu kỳ hoá: xen kẽ khối lượng, cường độ và phục hồi | Cấu trúc ba loại buổi mỗi tuần chính là chu kỳ hoá thu nhỏ. Cùng bài học: cường độ cao liên tục không bền, và nghỉ là một phần của kế hoạch chứ không phải sự thất bại.             |
| Âm nhạc             | Tập chậm từng đoạn khó, rồi ghép                    | Buổi chủ đề tương ứng với tập đoạn khó; buổi thi ảo tương ứng với chơi cả bản. Thiếu loại thứ hai thì bạn giỏi từng đoạn mà không chơi được cả bài.                                 |
| Y khoa, hàng không  | Rà soát sau ca / sau chuyến bay                     | Nhật ký lỗi là bản cá nhân của các quy trình đó. Điểm mạnh của bản thể chế: nó bắt buộc và không phụ thuộc tâm trạng — điều bạn có thể bắt buộc bằng cách gắn nó vào cuối mỗi buổi. |
| Học máy             | Học theo chương trình, dừng sớm, chọn mẫu khó       | Vùng luyện tương ứng với việc chọn mẫu có gradient hữu ích: quá dễ thì không học được gì, quá khó thì tín hiệu nhiễu. Khác biệt: bạn tự chọn được bài, mô hình thì không.           |
| Quản lý công việc   | Mục tiêu đo được, mốc kiểm tra định kỳ              | Cột “tiêu chí kiểm tra khách quan” trong bảng lộ trình chính là điều này. Không có tiêu chí thì “xong một tuần” trở thành “đã đọc hết”, vốn không đo được gì.                       |
| Nghiên cứu khoa học | Sổ thí nghiệm, bình duyệt                           | Danh sách mẫu là sổ ghi kết quả; thứ người tự học thiếu nhất là bình duyệt, và thay thế gần nhất là viết editorial công khai hoặc luyện theo cặp.                                   |

### TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì “nhiều” thường có nghĩa là nhiều bài *vừa sức*, mà vừa sức thì không tạo ra sai lệch dự đoán để học. Nó còn khuyến khích một thói quen tệ: đọc lời giải sớm để kịp chỉ tiêu số bài. Điều kiện đúng gồm bốn phần — mục tiêu hẹp, mức hơi quá sức, phản hồi nhanh, có sửa sai — và “giải nhiều” chỉ thoả phần cuối một cách hình thức. Chỉ số tốt hơn là số bài bạn giải xong *sau khi đã bí*.
- A2. 10 phút ôn (đọc nhật ký lỗi và một bảng active recall), 40 phút giải một bài ở vùng luyện có bấm giờ, 10 phút cho bước bốn — viết một câu mẫu và ghi nhật ký nếu sai. Nếu phải cắt, hãy cắt phần đọc kiến thức mới chứ đừng cắt 10 phút cuối, vì đó là phần biến một lần giải bài thành thứ dùng lại được. Và quan trọng hơn tỉ lệ là tính đều đặn: một giờ mỗi ngày trong sáu tháng hơn hẳn tám giờ mỗi cuối tuần.
- A3. Bảng ba chỉ số đo được. Tỉ lệ giải được ở một mức khó *cố định* — điểm mẫu chốt là mức phải cố định, nếu không bạn đang so hai thứ khác nhau. Thời gian trung bình cho một bài ở mức đã quen. Và số bài upsolve tới nơi mỗi tuần, chỉ số tốt nhất trong ba cái. Cảm giác chủ quan sai theo cả hai chiều: sau một buổi trôi chảy bạn thấy mình giỏi lên, còn khi gặp một chủ đề mới bạn thấy mình tệ đi — cả hai đều không phản ánh tiến bộ thật.

- A4. Khoảng một phần ba cho kiến thức mới, hai phần ba cho bài tập, và tỉ lệ nên dịch dần về phía bài tập theo thời gian. Lý do: kiến thức không có nhu cầu đi kèm sẽ bay hết trong hai tuần, còn bài tập vừa củng cố cái cũ vừa lộ ra cái còn thiếu — nên nó tự dẫn bạn tới nội dung cần học tiếp. Ngoại lệ: khi bắt đầu một chủ đề hoàn toàn mới, hãy dành nguyên một buổi cho kiến thức rồi cài thử ngay trong buổi đó.
- A5. Vì phần lớn lộ trình được thiết kế cho ngày lý tưởng chứ không cho ngày bận nhất. Một lịch đòi hai giờ mỗi ngày sẽ đứt ở tuần thứ ba, và cái đứt đó tốn hơn nhiều so với việc chỉ làm ba mươi phút mỗi ngày — vì xây lại thói quen khó hơn giữ nó. Cách thiết kế chống lại điều đó gồm ba phần: đặt mức tối thiểu mà bạn chắc chắn làm được kể cả ngày tệ nhất; làm cho tiến bộ nhìn thấy được bằng một bảng đánh dấu; và gắn buổi luyện vào một khung giờ cố định thay vì “khi nào rảnh”.

### NÓI LẠI THẬT ĐƠN GIẢN

Sau khi gấp sách lại, câu hỏi duy nhất còn lại là sáng mai làm gì. Câu trả lời gọn thế này. Mỗi buổi luyện, hãy dành mười phút đầu nhìn lại những lỗi cũ của mình, bốn mươi phút giải một bài đủ khó để bạn phải vật lộn, và mười phút cuối ghi lại hai thứ: ý chính của bài vừa giải viết thành một câu không nhắc tới đề bài, và nếu có sai thì sai vì sao. Mười phút cuối là phần dễ bị cắt nhất và cũng là phần tạo ra phần lớn tiến bộ.

Mỗi tuần, hãy có một buổi làm bài trộn nhiều chủ đề, có bấm giờ, như thi thật. Lý do: khi bạn luyện một chủ đề liên tiếp, bạn không phải *chọn* phương pháp — bạn biết trước bài này là quy hoạch động. Bài thật không nói cho bạn biết điều đó, nên kỹ năng chọn phải luyện riêng.

Và sau mỗi lần bí, hãy quay lại làm cho xong chứ đừng chỉ đọc lời giải rồi gật đầu. Cảm giác “à hoá ra thế” rất dễ nhầm với việc đã học được. Phép thử duy nhất đáng tin là đóng lời giải lại và tự viết từ trang trắng.

Cuối cùng, về khối lượng: đừng lên kế hoạch cho ngày đẹp trời. Hãy chọn một mức nhỏ mà bạn chắc chắn làm được kể cả hôm mệt nhất — ba mươi phút cũng được — rồi làm thêm khi rảnh. Lộ trình hồng gần như luôn vì lý do này, chứ không phải vì thiếu tài liệu hay thiếu bài tập.

*Một câu: Tiến bộ đến từ những bài bạn giải xong sau khi đã bí, không phải từ những bài bạn đọc lời giải.*

**Chỗ không rút gọn được:** Không có lịch luyện nào đúng cho mọi người; ba lộ trình ở đây là điểm xuất phát để bạn sửa theo dữ liệu của chính mình.

### Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

| Thuật ngữ / câu hỏi                       | Trả lời                                                                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| ●●● Bốn điều kiện chi phối mọi lịch luyện | Mục tiêu hẹp; mức hơi quá sức; phản hồi nhanh; có sửa sai. Thiếu một thì thời gian trôi mà năng lực đứng.                  |
| ●●● Cấu trúc một buổi 60 phút             | 10 phút ôn + 40 phút giải bài có bấm giờ + 10 phút viết câu mẫu và ghi nhật ký. Đừng cắt 10 phút cuối.                     |
| ●●● Ba loại buổi trong tuần               | Buổi kiến thức, buổi bài tập cùng chủ đề, buổi thi ảo trộn chủ đề. Tỉ lệ khởi đầu 2:3:1, dịch dần về phía thi.             |
| ●●● Vì sao phải trộn chủ đề               | Luyện tuần tự thì bạn không phải <i>chọn</i> phương pháp; bài thật không cho biết trước nên kỹ năng chọn phải luyện riêng. |
| ●●● Bốn công cụ theo dõi                  | Nhật ký lỗi; danh sách mẫu; lịch ôn ngắt quãng (1 ngày, 3 ngày, 1 tuần, 1 tháng); ba chỉ số tiến bộ.                       |
| ●●● Ba chỉ số tiến bộ                     | Tỉ lệ giải được ở mức khó <i>cố định</i> ; thời gian trung bình ở mức đã quen; số bài upsolve tới nơi mỗi tuần (tốt nhất). |
| ●● Phân loại nhật ký lỗi                  | Hiểu sai đề / sai ý tưởng / sai cài đặt. Tỉ lệ ba nhóm quyết định bạn nên luyện gì, và nó thường gây bất ngờ.              |
| ●● Tỉ lệ kiến thức và bài tập             | Khoảng 1:2, dịch dần về phía bài tập; kiến thức không có nhu cầu đi kèm sẽ quên trong hai tuần.                            |
| ●● Ôn nghĩa là gì                         | Tự truy xuất — che bảng active recall và tự trả lời, hoặc giải lại từ trang trắng. <i>Không</i> phải đọc lại.              |

| Thuật ngữ / câu hỏi             | Trả lời (tiếp)                                                                                                     |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------|
| ● ● Vì sao lộ trình đồ vỡ       | Thiết kế cho ngày lý tưởng thay vì ngày bận nhất. Đặt mức tối thiểu theo ngày tệ nhất, làm thêm khi rảnh.          |
| ● ● Tiêu chí “xong một tuần”    | Không phải đọc hết nội dung mà là đạt tiêu chí kiểm tra khách quan; chưa đạt thì kéo dài thay vì chạy tiếp.        |
| ● Nhịp duy trì cho người đi làm | Một bài mỗi tuần ở vùng luyện, một kỳ thi ảo mỗi tháng, đọc lại danh sách mẫu mỗi tháng — khoảng hai giờ mỗi tuần. |

● ● ● phải thuộc ● ● nên nắm ● biết là có

## I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

### 1. Tôi nên bắt đầu từ chương nào nếu đã biết cơ bản?

| Đọc A/01 và A/02 trước, dù đã biết — chúng quyết định cách bạn dùng phần còn lại. Sau đó vào thẳng phần C: C/15, C/16, C/17, C/19 là bốn chương có tỉ lệ ứng dụng cao nhất. Quay lại phần B khi một bài đòi hỏi một cấu trúc bạn chưa vững, và đọc E/25 sớm vì stress test là công cụ bạn nên có ngay từ đầu.

### 2. Làm sao chọn được bài ở đúng vùng luyện?

| Dùng hệ thống phân loại độ khó của nền tảng bạn dùng và chọn mức mà bạn giải được khoảng một nửa. Kiểm tra định kỳ: nếu ba buổi liên tiếp bạn giải hết trong dưới 15 phút thì nâng mức; nếu hai buổi liên tiếp không giải được gì và đọc lời giải vẫn mù mờ thì hạ. Với các tập bài có thứ tự như CSES[9], cứ đi tuần tự trong một chủ đề cũng đủ tạo độ khó tăng dần.

### 3. Tôi quên hết những gì đã học ba tháng trước. Làm sao?

| Đó là hiện tượng bình thường và có cách chữa cụ thể: ôn ngắt quãng bằng *truy xuất* chứ không phải đọc lại. Mỗi tuần, dành mười phút giải lại một bài cũ ngẫu nhiên từ trang trắng và che bảng active recall của một chương cũ để tự trả lời. Nếu bạn có danh sách mẫu, việc đọc lại nó mỗi tháng cũng rất hiệu quả vì mỗi câu mẫu là một khoá gợi nhớ cho cả một nhóm bài.

### 4. Nên dùng một ngôn ngữ hay nhiều ngôn ngữ?

| Một, và thành thạo nó. Trong thi đấu, việc gõ nhanh và không phải nghĩ về cú pháp là lợi thế thật; trong phỏng vấn, sự lúng túng về ngôn ngữ làm mất thời gian quý. Học ngôn ngữ thứ hai khi có lý do cụ thể — công việc yêu cầu, hoặc bạn cần một tính năng mà ngôn ngữ hiện tại không có. Điều đáng đầu tư hơn là hiểu sâu các cấu trúc dữ liệu chuẩn của ngôn ngữ bạn dùng, kể cả hằng số và hành vi ở trường hợp biên.

### 5. Tôi luyện một mình, không có ai đánh giá. Thay thế thế nào?

| Ba thứ thay thế được phần lớn vai trò của người hướng dẫn. Chấm bài tự động cho phản hồi đúng/sai tức thì. Stress test cho phản hồi ở đâu sai (E/25). Và việc viết editorial công khai hoặc giải thích cho người khác cho phản hồi về việc bạn có thực sự hiểu không — chỗ bạn viết không trôi chảy là chỗ bạn chưa hiểu. Nếu tìm được một bạn luyện cùng để phỏng vấn thử lẫn nhau thì đó là bổ sung có giá trị nhất.

### 6. Bao lâu thì thấy tiến bộ rõ rệt?

| Với nhịp 8–10 giờ mỗi tuần và luyện đúng cách, phần lớn người thấy khác biệt rõ sau khoảng 8–12 tuần — đủ để nâng một bậc độ khó. Nhưng có một giai đoạn khó chịu ở khoảng tuần thứ ba tới thứ sáu, khi bạn đã biết nhiều hơn nhưng chưa nhận diện đủ nhanh, nên cảm giác không tiến. Đó là lúc các chỉ số đo được ở mục 2 có giá trị nhất: chúng cho thấy tiến bộ mà cảm giác chưa thấy.

### 7. Sau khi đọc hết quyển sách này thì đọc gì tiếp?

| Chọn theo mục tiêu, và chỉ một quyển làm trực chính. Muốn nhận diện bài toán rộng hơn: sách của Skiena[10], đặc biệt nửa danh mục. Muốn học cách *thiết kế* thuật toán: Kleinberg–Tardos[11]. Muốn thi đấu: Laaksonen[12] và các bài tập kèm theo. Muốn đi sâu về chứng minh và đệ quy: Erick-

son[13]. Và quan trọng hơn mọi quyển sách: duy trì nhịp giải bài, vì đọc thêm mà không giải thì chỉ tăng cảm giác quen thuộc.

## Kết chương và kết sách

---

Quyển sách này bắt đầu bằng một quan sát ở chương A/01: khi bạn bí, cái thiếu hầu như không bao giờ là một thuật toán chưa từng nghe tên, mà là một quy trình để chạy khi chưa biết làm gì. Hai mươi tám chương sau, quy trình đó đã được cụ thể hoá: bốn bước để đối mặt với bài lạ, hai thước đo để tự chấm ý tưởng, một kho cấu trúc dữ liệu chọn theo thao tác cần làm rõ, tám khuôn tư duy để sinh lời giải, một ý thức về giới hạn để biết khi nào nên dừng, và một bộ kỹ năng nghề để đưa ý tưởng thành chương trình chạy được.

Nhưng phần quyết định không nằm trong sách. Nó nằm ở việc bạn có mở một bài ra vào sáng mai hay không, có viết một câu mẫu sau khi giải xong hay không, và có quay lại làm cho xong bài mình đã bí hay không. Ba việc đó tốn mười phút mỗi ngày và chúng là toàn bộ khác biệt giữa người đọc xong một quyển sách và người giỏi lên.

Chúc bạn luyện đều, bí nhiều, và mỗi lần bí đều học được một điều mới.



# Nguồn

---

- [1] A. Zeller and R. Hildebrandt, "Simplifying and Isolating Failure-Inducing Input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [2] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [3] D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [4] J. L. Bentley and M. D. McIlroy, "Engineering a Sort Function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [5] D. R. Musser, "Introspective Sorting and Selection Algorithms," *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [6] J. Bentley, *Writing Efficient Programs*. Prentice-Hall, 1982.
- [7] F. McSherry, M. Isard, and D. G. Murray, "Scalability! But at What COST?" In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [8] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [9] A. Laaksonen. "CSES Problem Set." (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [10] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [11] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [12] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [13] J. Erickson, *Algorithms*. Independently published, 2019.

## Nguồn

---

- [1] A. Zeller and R. Hildebrandt, "Simplifying and Isolating Failure-Inducing Input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [2] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [3] D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [4] J. L. Bentley and M. D. McIlroy, "Engineering a Sort Function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [5] D. R. Musser, "Introspective Sorting and Selection Algorithms," *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [6] J. Bentley, *Writing Efficient Programs*. Prentice-Hall, 1982.
- [7] F. McSherry, M. Isard, and D. G. Murray, "Scalability! But at What COST?" In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [8] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [9] A. Laaksonen. "CSES Problem Set." (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [10] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [11] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [12] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [13] J. Erickson, *Algorithms*. Independently published, 2019.